Transfer Learning-лабораторія 3.1: посібник користувача

Версія 3.1 · жовтень 2026 · навчальна програма до заняття 4.13 «Практичне використання репозиторіїв аналітичних моделей» (НУОУ)

1. Що це за програма і навіщо

Transfer Learning-лабораторія — портативний навчальний застосунок: шість незалежних модулів, у кожному **готова модель із відкритого репозиторію** пристосовується до військово-прикладної навчальної задачі й чесно порівнюється з базовим рівнем. Встановлюється однією командою, після встановлення працює **без Інтернету** і переноситься на флешці.

Головна ідея — transfer learning (перенесення навчання). Навчити нейромережу з нуля — це мільйони прикладів, тижні обчислень і дорогі відеокарти. Готова модель уже «знає» загальні речі: як виглядають об'єкти, як звучать двигуни, що означають слова. Ми беремо її з репозиторію і доучуємо лише невелику частину на кількох десятках прикладів — за хвилини на звичайному ноутбуці. Саме так ШІ можна швидко впроваджувати в підрозділах.

Модуль	Задача	Готова модель	Що робимо
01	Фото: БПЛА, бронетехніка, артилерія, вантажівка, замасковані позиції, немає цілі	ResNet18 (ImageNet)	новий останній шар + аугментація
02	Документи: донесення, повідомлення, довідка, наказ, рапорт, заявка	MiniLM-L12	класифікатор на ембеддингах
03	Звуки: Шахед, реактивний Шахед, вертоліт, літак, техніка, бензопила, сирена, фон	YAMNet (AudioSet)	класифікатор на ембеддингах
04	Групування потоку повідомлень за темами без міток	MiniLM + KMeans	готові ознаки, без донавчання
05	Тестові запитання за навчальним текстом, експорт у Word і Moodle	Qwen3-0.6B	few-shot, RAG, LoRA
06	Агент керування в нових умовах (фізика, вітер, шум датчиків)	PPO CartPole / Acrobot	донавчання + контроль забування

Кожен модуль має однакову структуру: **Модель і дані → Адаптація → Оцінювання → Демонстрація → Теорія**. Незнайомі терміни в програмі підкреслено — наведіть мишу, і з'явиться пояснення; повний перелік — у **«Словнику понять»** (розділ 13 цього посібника).



Ситуаційна панель: стан шести модулів і останні результати

2. Як розповісти про програму за 5 хвилин (шпаргалка доповідача)

Тези, які варто сказати:

1. «Ми не навчали нейромережі з нуля — ми взяли готові моделі з відкритих репозиторіїв (PyTorch, Hugging Face, TensorFlow Hub) і пристосували їх до своїх задач. Це і є transfer learning».
2. «Готова модель уже вміє головне: бачити форми, чути звуки, розуміти зміст. Ми навчаємо лише "останню милю" — тисячі параметрів замість мільйонів. Тому вистачає 20–30 прикладів на клас і кількох хвилин на ноутбуці».
3. «Кожен результат чесно порівняний з базовим рівнем — простим методом без адаптації. Табло "до → після" показує, скільки дала саме адаптація».
4. «Модель — помічник, а не той, хто приймає рішення. Тому програма пояснює свої рішення (Grad-CAM, найближчі приклади) і чесно каже "Невизначено", коли не впевнена».
5. «Усі дані — відкриті або синтетичні; програма працює офлайн і нічого не надсилає в мережу».

Прості аналогії для кожного модуля:

Модуль	Аналогія для слухачів
01	Курсант, що розрізняє автомобілі, за вечір навчиться відрізняти БТР від САУ — бачити колеса й башти він уже вміє
02	«Станом на 08:00 рота утримує рубіж» і «за даними на ранок підрозділ тримає позиції» — різні слова, один зміст

Модуль	Аналогія для слухачів
03	Досвідчений боєць на слух відрізняє Шахед від мотоцикла — модель «чула» 2 млн звуків і вчиться наших назв
04	Розкласти 60 записок на купки «за змістом», не знаючи заздалегідь, які купки бувають
05	Новий писар пише рапорт за трьома зразками (few-shot), за найсхожішими зразками (RAG) чи після тренування (LoRA)
06	Водій легковика пересідає на вантажівку: спершу незграбно, за кілька поїздок пристосовується — і не забуває легковик

Сценарій на 5 хвилин: 0:00–0:45 — задача й навіщо вона підрозділу; 0:45–1:45 — яку готову модель узяли і що в ній навчали (цифри — у «Теорії» модуля); 1:45–2:45 — жива демонстрація в режимі «Доповідь»; 2:45–3:45 — табло «до → після» і базовий рівень; 3:45–4:30 — типові помилки й обмеження; 4:30–5:00 — висновок: чи можна застосовувати і з якими застереженнями.

3. Установка однією командою

Потрібно: Windows 10/11 ×64, ≈ 9 ГБ вільного місця, Інтернет **лише на час установки** (20–60 хв).

1. Розпакуйте `TL-Lab.zip` у коротку папку без кирилиці, наприклад `C:\TL-Lab` (кирилиця й довгі шляхи заважають частині бібліотек).
2. Двічі клацніть **`INSTALL.bat`**. Далі все автоматично — 8 кроків:

Крок	Що відбувається	Обсяг	Орієнтовно
1. Перевірка системи	64-розрядна Windows, вільне місце, шлях, Інтернет	—	секунди
2. Python 3.11	вбудований Python у `runtime\python` (системний не чіпається)	≈ 25 МБ	1 хв
3. Бібліотеки	PyTorch, TensorFlow, Transformers, FastAPI… точно за списком версій	≈ 2 ГБ	10–30 хв
4. Готові моделі	ResNet18, MiniLM, YAMNet, Qwen3-0.6B, агенти PPO CartPole і Acrobot	≈ 2 ГБ	5–20 хв
5. Фото	до 30 фото на кожен із 6 класів з Wikimedia Commons + автори й ліцензії	≈ 20 МБ	3–8 хв
6. Звуки	Шахеди — з Wikimedia Commons, інші класи — з ESC-50	≈ 30 МБ	2–6 хв
7. Самоперевірка	бібліотеки, моделі, дані кожного модуля	—	секунди
8. Ярлик	«Transfer Learning-лабораторія» на робочому столі	—	секунди

3. Якщо Інтернет обірвався — просто запустіть `INSTALL.bat` ще раз: виконані кроки пропускаються, докачується лише відсутнє.

Додаткові параметри (запуск з командного рядка в папці TL-Lab): `INSTALL.bat -PhotosPerClass 60` — більше фото; `-SoundsPerClass 40` — більше звуків; `-SkipPhotos`, `-SkipSounds` — без збору даних; `-Repair` — перевстановити бібліотеки.

Оновлення з версії 2.x: запустіть `TL-Lab-Stop.bat`, розпакуйте новий архів поверх старої папки (із заміною файлів) і запустіть `INSTALL.bat` — він доставить нові бібліотеки, агента Acrobot, фото нових класів і звуки. Ваші фото, моделі й результати залишаться.

> Після автоматичного збору **обов'язково перегляньте фото і прослухайте звуки** в «Центрі даних» і видаліть невідповідні — особливо в класах «Замасковані позиції» і «Шахед» (розділ 11).

4. Запуск, зупинка, перенесення

- **Запуск** — ярлик «Transfer Learning-лабораторія» або `TL-Lab.bat`. Через 5–15 с відкриється вікно програми (Edge у режимі застосунку або браузер за замовчуванням, адреса `http://127.0.0.1:8501`). Інтернет не потрібен.
- **Чорного вікна немає.** Сервер програми працює у фоні, невидимо; на екрані лише вікно самої програми.
- **Вимкнення** — кнопка **«⏻ Вимкнути програму»** внизу лівої панелі або в «Налаштуваннях» (натиснути двічі: друге натискання — підтвердження), або файл `TL-Lab-Stop.bat`. Якщо просто закрити вікно програми, сервер вимкнеться сам через 15 хв, коли немає активних задач (повторне відкриття протягом цього часу — миттєве).
- **Перенесення** — папку `TL-Lab` можна цілком скопіювати на флешку чи інший ПК з Windows 10/11 ×64: нічого встановлювати не потрібно.
- **Журнали** — `app\logs\server.err.log` і `server.log` (сервер; якщо програма аварійно закрилася, причина тут), `app\logs\app.log`, `app\logs\qwen_worker.log` (модуль 05).

Структура папки

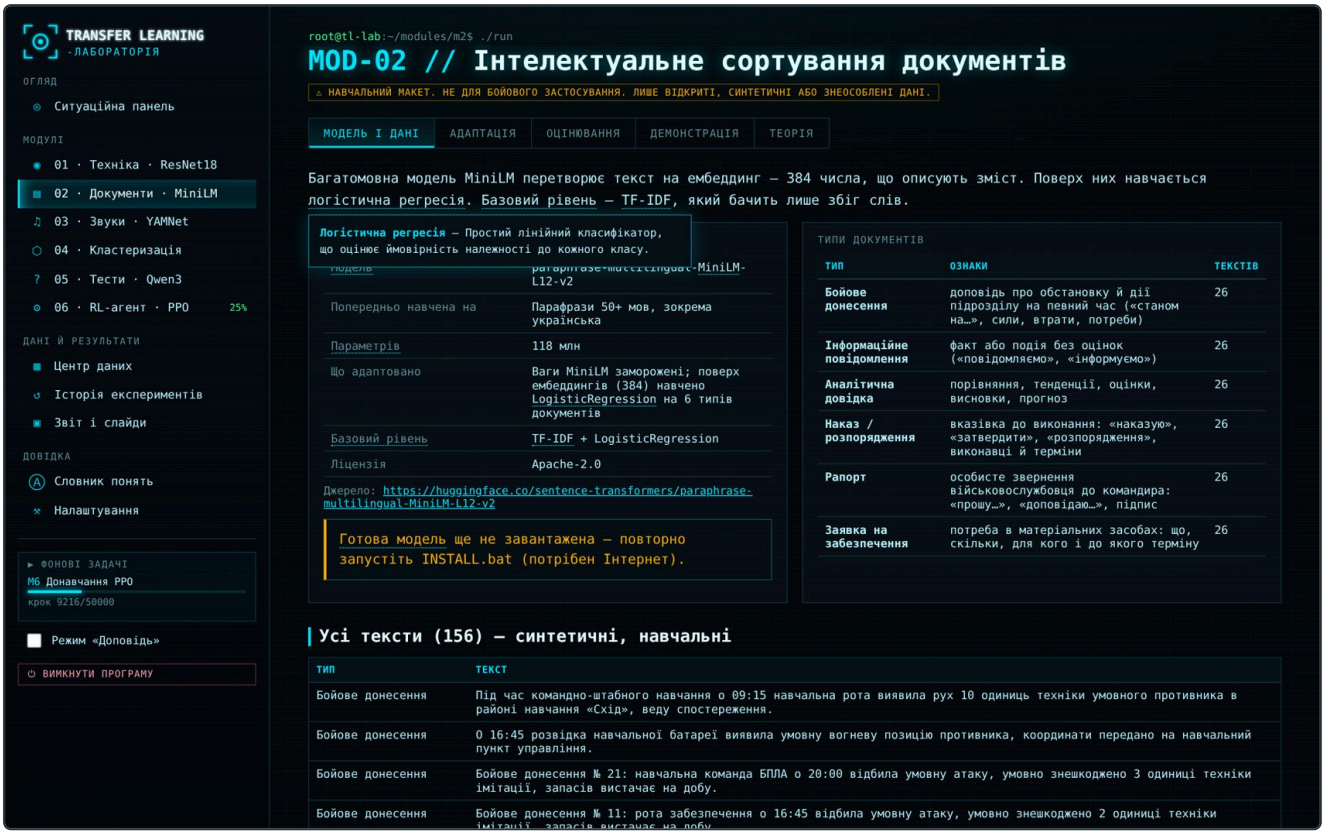
```
TL-Lab\
├─ INSTALL.bat           установка (один раз, з Інтернетом)
├─ TL-Lab.bat            запуск
├─ TL-Lab-Stop.bat       вимкнення (якщо кнопка в програмі недоступна)
├─ ПРОЧИТАЙ.txt          коротка інструкція
├─ install.log           журнал установки
├─ runtime\python\       вбудований Python і бібліотеки (≈ 4 ГБ)
└─ app\
   ├─ server\            сервер програми (FastAPI)
   ├─ web\               інтерфейс (HTML, CSS, JavaScript; працює офлайн)
   ├─ modules\           логіка шести модулів, теорія, словник
   ├─ data\              фото, звуки, тексти, джерела й ліцензії
   ├─ models\            готові моделі й ваші адаптовані моделі
   ├─ results\           журнал експериментів, графіки, відео
   ├─ logs\              журнали
   └─ notebooks\         блокнот Google Colab для LoRA
```

5. Інтерфейс

- **Ліворуч** — навігація: Ситуаційна панель, шість модулів, Центр даних, Історія експериментів, Звіт і слайди, Словник понять, Налаштування. Сторінки перемикаються миттєво, без перезавантаження.
- **Розділи модуля** — кнопки під заголовком: «Модель і дані», «Адаптація», «Оцінювання», «Демонстрація», «Теорія».
- **Фонові задачі.** Навчання, оцінка, збір даних виконуються у фоні: можна вільно переходити між модулями й запускати інші задачі. Прогрес видно в лівій панелі («▶ Фонові задачі») і на сторінці модуля: смуга, консоль подій, кнопка «Скасувати». Після завершення з'являється сповіщення, а модуль сам відкриває «Оцінювання».
- **Черга.** За замовчуванням виконується одна задача навчання за раз (решта в черзі) — так моделі не ділять процесор і пам'ять. На ПК з 16 ГБ ОЗП можна дозволити дві паралельні (Налаштування → Продуктивність).
- **Підказки.** Підкреслені терміни — наведіть мишу, щоб побачити пояснення; клік відкриває словник.
- **Режим «Доповідь»** (перемикач унизу ліворуч) — великі табло «до → після», службові параметри приховано (кнопки запуску лишаються).
- **«Що відбувається під час адаптації — крок за кроком»** — на вкладці «Адаптація» кожного модуля: простими словами, що робить програма після натискання кнопки.
- **«Чому так вийшло»** — на вкладці «Оцінювання»: програма сама пояснює результат за цифрами останнього запуску — що покращилось і на скільки, з чим модель плутає і чому, які класи слабкі, що спробувати далі. Окремо вона перевіряє, чи різниця **не випадковість**: на тесті з n прикладів і точністю p результат «гуляє» приблизно на $\pm 2\sqrt{p(1-p)n}$ прикладів. Наприклад, 144 фото і 90 %: ±7 фото, тож 90 % → 89 % — це нічия, а не погіршення.
- **«ⓘ Як читати цей графік»** — під кожним графіком і табло: що по осях, де «добре», де «погано».
- **Заморожена чи розморожена** — у модулях 02, 03 (експериментально) і 06 можна додатково донавчити верхні шари самої мережі (або, в модулі 06, навпаки — лише останній шар) і побачити порівняння в одній таблиці.



Фонове донавчання: прогрес, журнал подій, «Скасувати»; ліворуч — монітор задач



Підказка до терміна прямо в тексті модуля

Налаштування (розділ «Інтерфейс»): анімація фону («Цифровий дощ», «Радар» або «Вимкнено»), її інтенсивність і пауза під час навчання, плавні переходи, режим «Доповідь». «Модулі»: поріг «Невизначено» (модуль 01), кроки донавчання (модуль 06). «Продуктивність»:

кількість паралельних задач, прогрів бібліотек. «Дані й кеш»: розміри папок, очищення. «Система»: версії бібліотек і стан кожного модуля.

6. Модуль 01 — Класифікатор техніки й позицій на фото

Готова модель: ResNet18 (ImageNet).

Навіщо це військовим

Оператор БПЛА чи аналітик отримує сотні знімків за зміну. Модель робить перший прохід: позначає, де на фото техніка, артилерія чи щось схоже на замасковану позицію, а де «порожня» місцевість. Людина перевіряє лише відібране — і не пропускає головного через втому. Остаточне рішення завжди за людиною.

Простий приклад

Курсант, який уже вміє розрізняти автомобілі, за один вечір навчиться відрізняти БТР від САУ — йому не треба заново вчитися бачити колеса й башти. Так само ResNet18 уже «бачила» 1,28 млн фото: ми показуємо їй 20–30 знімків кожного класу і вчимо лише останній шар — «як назвати те, що вона бачить».

Як працювати

1. **Модель і дані** — перевірте, скільки фото в кожному з 6 класів. Класи з менш ніж 5 фото не навчаються; рекомендовано 20–30. Бракує — «Центр даних → Фото → Зібрати фото для всіх класів».
2. **Адаптація** — залиште 10 епох і lr = 0,001 → «Запустити адаптацію». Спершу навчається базовий варіант (без аугментації), потім адаптований; на CPU 5–15 хв. Можна переходити до інших модулів — навчання йде у фоні.
3. **Оцінювання** — табло «до → після» (accuracy, F1-macro), «Чому так вийшло» (наприклад: «90 % → 90 % — різниця 1 фото з 144, це випадковість»), дві матриці помилок, крива втрат, помилки з тепловими картами Grad-CAM: чи «дивилась» мережа на об'єкт, чи на фон.
4. **Демонстрація** — клацніть демо-фото або перетягніть своє → клас, впевненість, Grad-CAM. Нижче порогу (60 %, змінюється в налаштуваннях) програма чесно пише «Невизначено».

Що показати на доповіді

Покажіть одне правильне розпізнавання з Grad-CAM на об'єкті і одну помилку, де «гаряча» зона — на фоні. Скажіть: «Модель навчилась на 30 фото на клас, бо 11,7 млн ваг уже навчені на ImageNet — ми вчили лише 3 тисячі».



Розділ «Теорія»: формули й пояснення прямо в програмі

Теорія

ІДЕЯ

Transfer learning (перенесення навчання). Згорткова мережа ResNet18 уже навчена розпізнавати 1000 класів ImageNet (1,28 млн фото). Її ранні шари виділяють універсальні ознаки — краї, текстури, форми, частини об'єктів. Ці ознаки корисні й для техніки, тому ми **не навчаємо мережу з нуля**, а замінюємо лише останній шар («голову») на новий — на 6 наших класів — і навчаємо тільки його.

Класи: БПЛА, бронетехніка, артилерія, вантажівка, замасковані позиції, «немає цілі» (порожня місцевість). Останній клас важливий: без нього модель на будь-якому фото «бачила» б техніку. Класи з менш ніж 5 фото автоматично пропускаються під час навчання (з попередженням у журналі).

АРХІТЕКТУРА І ВАГИ

- **ResNet18** (He et al., 2015): 18 шарів з вагами, 4 стадії по 2 залишкові блоки (64 → 128 → 256 → 512 каналів), вхід 224×224×3, наприкінці global average pooling → вектор із 512 ознак → повнозв'язний шар `fc`.
- Ваги **IMAGENET1K_V1** з torchvision (точність top-1 на ImageNet ≈ 69,8 %), усього **11 689 512** параметрів.
- Ми заморожуємо всі згорткові шари і ставимо новий `fc` 512 → 6: навчаються лише **3 078** параметрів (512·6 ваг + 6 зсувів). Опція «Розморозити layer4» додає ще **8 393 728** параметрів останньої стадії.

Залишковий блок навчає не саме перетворення, а «поправку» до входу — це дозволяє тренувати глибокі мережі без загасання градієнта:

$$y = \mathcal{F}(x, \{W_i\}) + x$$

ФОРМУЛИ НАВЧАННЯ

Новий шар перетворює 512 ознак h на 6 оцінок (логітів) і далі на ймовірності функцією softmax:

$$z = Wh + b, \qquad p_k = \frac{e^{z_k}}{\sum_{j=1}^{6} e^{z_j}}$$

Функція втрат — перехресна ентропія (штраф тим більший, чим менша ймовірність правильного класу y):

$$\mathcal{L} = -\log p_y$$

Оптимізатор **Adam** (lr = 0,001, батч 16, 10 епох) оновлює лише ваги W, b.

Аугментація — випадкове обрізання (60–100 % кадру), віддзеркалення, зміна яскравості/контрасту/насиченості ±30 %. Кожна епоха бачить «нові» варіанти тих самих фото, що зменшує перенавчання на 20–30 знімках на клас.

ПОЯСНЕННЯ РІШЕНЬ: GRAD-CAM

Grad-CAM (Selvaraju et al., 2017) показує, на які ділянки фото спиралася мережа. Для класу c беруться активації A^k останнього згорткового блоку (512 карт 7×7) і градієнти оцінки y^c за ними:

$$\alpha_k^c = \frac{1}{Z}\sum_i\sum_j \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L^c_{\text{Grad-CAM}} = \mathrm{ReLU}\Big(\sum_k \alpha_k^c A^k\Big)$$

Карта 7×7 збільшується до 224×224 і накладається на фото. Якщо «гаряча» зона — небо чи дорога, а не об'єкт, модель навчилася на фоні: це типова причина помилок.

ОЦІНЮВАННЯ

- Поділ даних **70/30 зі стратифікацією** (частки класів однакові в обох вибірках), seed 42.
- **Базовий рівень** — той самий класифікатор без аугментації: різниця показує внесок аугментації. Випадкове вгадування для 6 класів дає ≈ 17 %.
- Метрики: **accuracy** (частка правильних) і **F1-macro** — середнє F1 по класах:

$$P = \frac{TP}{TP+FP}, \quad R = \frac{TP}{TP+FN}, \quad F_1 = \frac{2PR}{P+R}, \quad F_1^{macro} = \frac{1}{K}\sum_{k=1}^{K} F_1^{(k)}$$

- Якщо максимальна ймовірність < 0,6, програма пише «Невизначено» — краще чесне «не знаю», ніж впевнена помилка.

ЧОМУ ТАК

- 11,7 млн параметрів неможливо навчити на ~150 фото — мережа просто запам'ятає їх. 3 078 параметрів «голови» — можна.
- Ознаки ImageNet містять транспорт і гармати (танки, вантажівки, літаки), тому добре переносяться на нашу задачу.
- Замасковані позиції — найскладніший клас: контури розмиті, кольори як у фону. Тут модель найчастіше помиляється — це чесно показує межі методу.

- Дублікати фото в навчальній і тестовій вибірках штучно завищують точність — тому є пошук і видалення дублікатів (aHash).

7. Модуль 02 — Інтелектуальне сортування документів

Готова модель: MiniLM-L12 (багатомовна).

Навіщо це військовим

У штаб надходять десятки документів за годину. Автоматичне сортування за типом (донесення, наказ, рапорт, заявка, довідка, повідомлення) скорочує час, поки документ потрапить до потрібного виконавця, і допомагає не загубити термінове донесення серед заявок на пальне.

Простий приклад

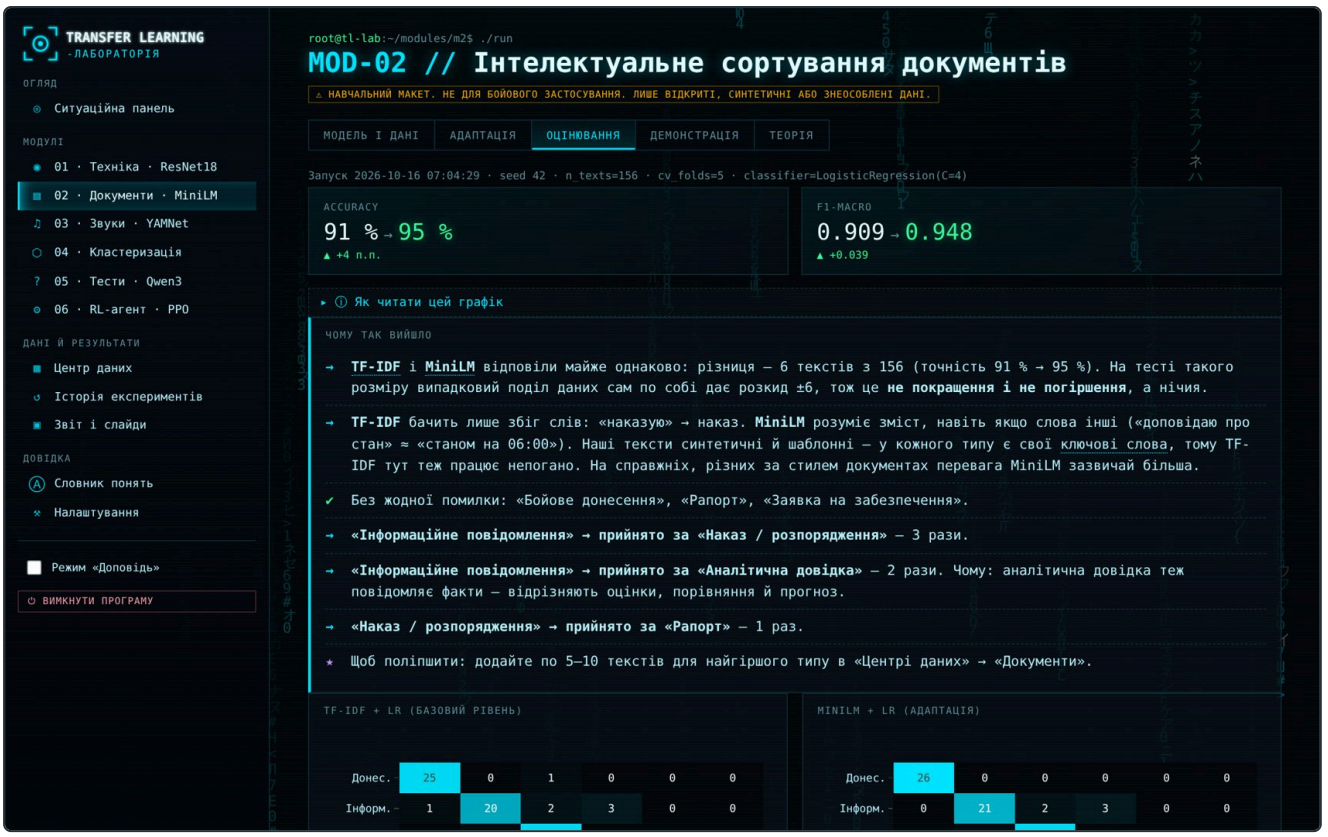
«Станом на 08:00 рота утримує рубіж...» і «За даними на ранок підрозділ тримає позиції...» — слова різні, а зміст один. Пошук за словами (TF-IDF) їх не зв'яже, а ембеддинги MiniLM поставлять поруч. Тому MiniLM розрізняє документи за змістом, а не за збігом слів.

Як працювати

1. **Модель і дані** — 6 типів документів по 26 синтетичних зразків; таблиця ознак кожного типу.
2. **Адаптація** — «Запустити адаптацію»: тексти кодуються MiniLM (перший раз ~30 с, далі з кешу), потім 5-кратна крос-валідація обох методів.
3. **Розморожування (опція)** — галочка «Також розморозити останні шари MiniLM»: оберіть 1–4 шари з 12, епохи, learning rate. Додає 3–10 хв; порівняння — на тих самих фолдах.
4. **Оцінювання** — табло TF-IDF → MiniLM, «Чому так вийшло», таблиця «заморожена / розморожена» (якщо вмикали), матриці помилок, хибні відповіді з колонкою «Чому могло статися».
5. **Демонстрація** — оберіть приклад або вставте свій текст → тип, ймовірності й 3 найближчі навчальні тексти («чому модель так вирішила»).

Що показати на доповіді

Вставте два тексти з однаковим змістом різними словами — MiniLM віднесе їх до одного типу. Ключова фраза: «TF-IDF рахує слова, MiniLM розуміє зміст».



Оцінювання: «до → після» і матриці помилок TF-IDF та MiniLM

Теорія

ІДЕЯ

Текст перетворюється на **ембеддинг** — вектор із 384 чисел, у якому близькі за змістом тексти лежать поруч, навіть якщо в них різні слова. Готова мовна модель MiniLM робить це «з коробки»; ми лише вчимо простий класифікатор розрізняти шість типів документів у цьому просторі.

Набір — 156 синтетичних навчальних документів (по 26 кожного типу): бойове донесення, інформаційне повідомлення, аналітична довідка, наказ / розпорядження, рапорт, заявка на забезпечення.

АРХІТЕКТУРА І ВАГИ

- **paraphrase-multilingual-MiniLM-L12-v2** (Sentence-Transformers): 12 шарів трансформера, розмір прихованого стану 384, 12 голів уваги, ≈ 118 млн параметрів (більшість — словник на 250 тис. токенів для 50+ мов).
- Навчена дистиляцією знань (Reimers & Gurevych, 2020): багатомовна «учениця» вчиться давати ті самі вектори, що й англомовна «вчителька» для перекладів одного речення — тому українська працює.
- **Ваги MiniLM заморожені.** Навчається лише логістична регресія: 6 × 384 + 6 = **2 310** параметрів.

ФОРМУЛИ

Ембеддинг речення — середнє векторів його токенів (mean pooling) з L2-нормалізацією:

$$u = \frac{1}{n}\sum_{i=1}^{n} h_i, \qquad \hat{u} = \frac{u}{\|u\|}$$

Схожість двох текстів — косинус кута між векторами:

$$\cos(u, v) = \frac{u \cdot v}{\|u\| \, \|v\|}$$

Логістична регресія (мультиноміальна) з L2-регуляризацією, $C = 4$:

$$P(y = k \mid u) = \mathrm{softmax}(Wu + b)_k, \qquad \min_{W,b} \; -\sum_i \log P(y_i \mid u_i) + \frac{1}{2C}\|W\|^2$$

Базовий рівень TF-IDF зважує слова й біграми: часто в документі, рідко в корпусі — важливо:

$$\mathrm{tfidf}(t, d) = \big(1 + \log \mathrm{tf}(t, d)\big) \cdot \left(\log \frac{1 + N}{1 + \mathrm{df}(t)} + 1\right)$$

ОЦІНЮВАННЯ

- **5-кратна стратифікована крос-валідація**: 156 текстів ділять на 5 частин; кожна частина по черзі є тестовою, а модель навчається на решті. Так кожен текст отримує прогноз від моделі, яка його не бачила, і оцінка стабільніша, ніж на одному малому тесті.
- Метрики: accuracy, F1-macro, матриця помилок; обидва методи — на тих самих фолдах.
- **Пояснення рішення**: для нового тексту показуються 3 найближчі навчальні тексти за косинусною схожістю.

ЧОМУ ТАК

- TF-IDF бачить лише збіг слів: «станом на 08:00» і «за даними на ранок» для нього різні. MiniLM — схожі.
- 156 прикладів замало для донавчання трансформера на 118 млн параметрів; лінійний класифікатор на готових ембеддингах — оптимальний вибір для малих даних.
- Складні межі — «аналітична довідка» з фактами схожа на «інформаційне повідомлення», а «бойове донесення» — на «рапорт-доповідь»; це видно в матриці помилок.
- Синтетичні тексти шаблонні, тому оцінка на них оптимістична; на реальних документах точність буде нижчою.

РОЗМОРОЖУВАННЯ MINILM (ОПЦІЯ)

Заморожена MiniLM (звичайна адаптація): усі 118 млн ваг незмінні, навчається лише логістична регресія — 2 310 чисел.

Розморожена: останні L з 12 шарів трансформера теж навчаються разом із лінійною «головою». Решта шарів заморожені — вони вже добре розуміють мову.

$$\theta_{\text{нав}} = \theta_{\text{голова}} \cup \theta_{\text{шари } 12-L+1 \ldots 12}$$

Щоб перші кроки не зіпсували готові ознаки, голова стартує з уже навченої логістичної регресії (**LP-FT**), а learning rate для шарів малий (10^{-5}–10^{-4}). Порівняння — на тих самих фолдах крос-валідації, тож чесне.

Чого очікувати: на наших 156 шаблонних текстах замороженої моделі зазвичай досить — приріст малий або нульовий. Це нормальний і важливий висновок: розморожувати варто, коли даних багато або вони дуже «свої».

8. Модуль 03 — Розпізнавання звуків: Шахеди, авіація, техніка

Готова модель: YAMNet (AudioSet).

Навіщо це військовим

Акустичні датчики — дешеве доповнення до радарів для виявлення низьколітних ударних БПЛА. Модель розрізняє звук поршневого Шахеда («мопед»), реактивного Шахеда, вертольота, літака, наземної техніки й фонові шуми та допомагає оператору швидше відреагувати на загрозу.

Простий приклад

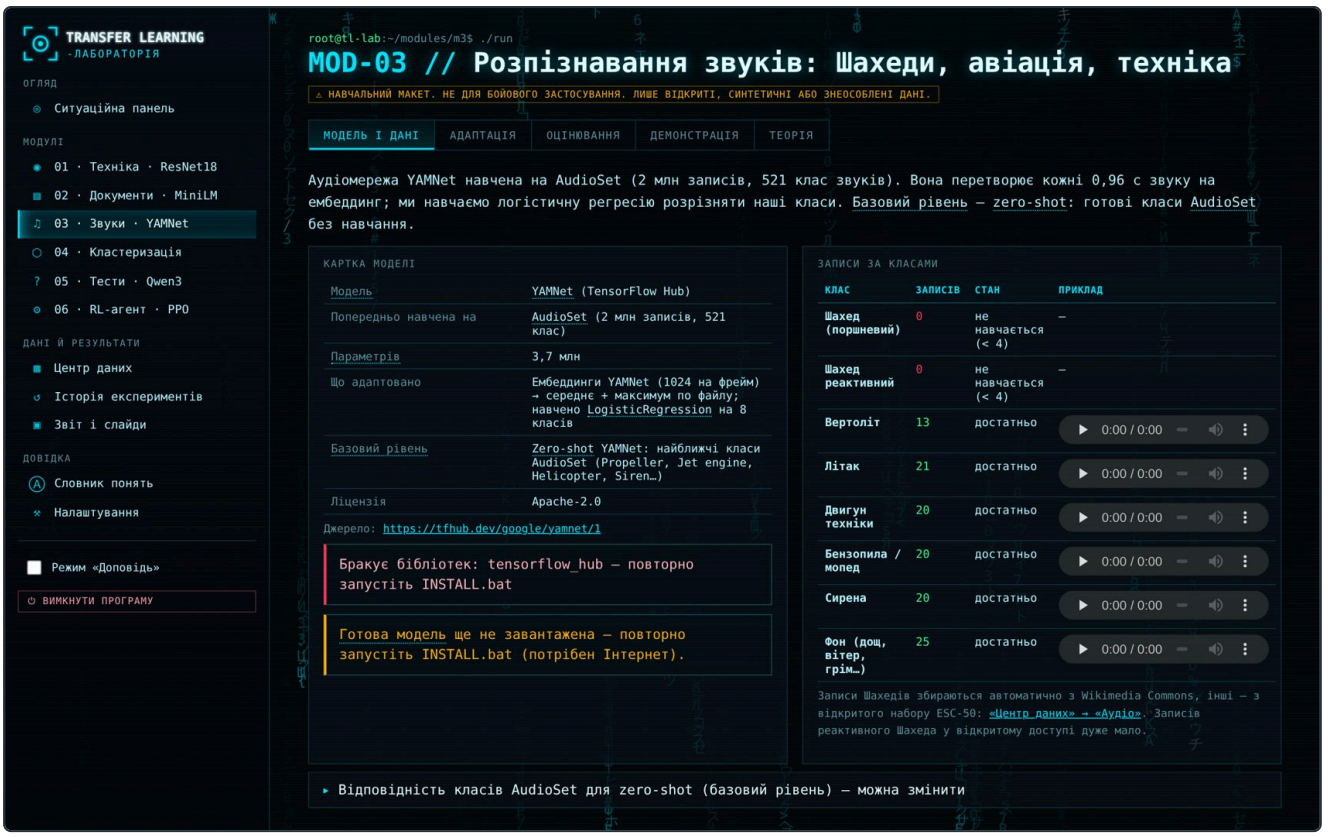
Досвідчений боєць на слух відрізняє Шахед від мотоцикла. Готова мережа YAMNet «чула» 2 млн звуків і вже знає, як звучать двигуни, гвинти й сирени, але не знає слова «Шахед». Ми даємо їй кілька десятків записів кожного класу — і вона вчиться називати знайоме звучання нашими назвами.

Як працювати

1. **Модель і дані** — таблиця 8 класів: скільки записів, стан («достатньо», «мало даних», «не навчається»), приклад для прослуховування. Записи Шахедів докачуються в «Центрі даних → Аудіо → Зібрати звуки».
2. **Адаптація** — «Запустити адаптацію». YAMNet працює в окремому процесі: перший запуск — до 20 с на старт TensorFlow.
3. **Експеримент (опція)** — «також розморозити верхні блоки YAMNet»: записів мало, тож результат може бути гіршим — це теж висновок для доповіді (перенавчання). Якщо версія моделі не дозволяє змінювати ваги, програма повідомить.
4. **Оцінювання** — zero-shot → YAMNet + LR, «Чому так вийшло», матриці помилок, записи з помилками (можна прослухати) з колонкою «Чому могло статися».
5. **Демонстрація** — ▶ біля запису або свій файл → клас, zero-shot, топ-5 AudioSet, хвиля і мел-спектрограма.

Що показати на доповіді

Увімкніть запис бензопили і запис Шахеда: на слух схожі, а модель розрізняє. Чесно назвіть обмеження: відкритих записів Шахедів мало, реактивних — одиниці.



Модуль 03: класи звуків і стан даних («мало даних» — червоним)



Оцінювання з блоком «Чому так вийшло» (приклад на тестових даних)

Теорія

ІДЕЯ

Звук перетворюється на спектрограму, а готова аудіомережа YAMNet, навчена на AudioSet, видає для кожного фрагмента 0,96 с вектор ознак. Ми порівнюємо два підходи: **zero-shot** (беремо найближчі готові класи AudioSet — Propeller, Jet engine, Helicopter, Siren… — без навчання) і **transfer learning** (навчаємо свій класифікатор на ознаках YAMNet).

Класи: Шахед (поршневий), Шахед реактивний, вертоліт, літак, двигун техніки, бензопила / мопед (схожий на Шахед звук), сирена, фон (дощ, вітер, гроза, поїзд, феєрверк). Записи Шахедів збираються автоматично з Wikimedia Commons (вирізаються фрагменти зі сталим гулом), решта — з відкритого набору ESC-50. Класи з менш ніж 10 записами позначаються «мало даних», з менш ніж 4 — не навчаються.

АРХІТЕКТУРА І ВАГИ

- **YAMNet** (Google, TensorFlow Hub): архітектура MobileNetV1 з depthwise-separable згортками, ≈ 3,7 млн параметрів, навчена на **AudioSet** (≈ 2 млн 10-секундних відео YouTube, 521 клас звуків).
- Вхід — хвиля 16 кГц моно. Усередині: STFT (вікно 25 мс, крок 10 мс) → 64 мел-смуги 125–7500 Гц → логарифм → патчі 96 × 64 (0,96 с) з кроком 0,48 с.
- Вихід на кожен патч: 521 ймовірність класів і **ембеддинг 1024**. Ваги YAMNet заморожені; навчається лише логістична регресія: 8 × 2048 + 8 = **16 392** параметри.
- YAMNet (TensorFlow) працює в **окремому процесі** програми: TensorFlow і PyTorch в одному процесі конфліктують.

Depthwise-separable згортка замінює звичайну згортку $D_K \times D_K$ з N фільтрами на дві дешеві операції, скорочуючи обчислення приблизно в

$$\left(\frac{1}{N} + \frac{1}{D_K^2}\right)^{-1} \text{ разів } (\approx 8\text{–}9 \text{ для } 3 \times 3)$$

ФОРМУЛИ

Мел-шкала наближає частоту до сприйняття людиною:

$$m = 2595 \, \log_{10}\left(1 + \frac{f}{700}\right)$$

Ознаки файлу з T патчами — середнє й максимум ембеддингів (2048 чисел):

$$\phi = \left[\tfrac{1}{T}\sum_t e_t \,\middle\|\, \max_t e_t\right]$$

Zero-shot: оцінка класу c — найбільша із середніх за часом ймовірностей класів AudioSet A_c, що йому відповідають (максимум, а не сума, щоб клас із довшим списком назв не мав переваги):

$$s_c = \max_{a \in A_c} \frac{1}{T}\sum_t p_{t,a}, \qquad \hat{y} = \arg\max_c s_c$$

Відбір фрагментів Шахеда з відео: вікна по 5 с, де гучність висока, а **пік-фактор** (максимум / середньоквадратичне) малий — сталий гул двигуна проходить, вибухи й постріли відкидаються:

$$\text{crest} = \frac{\max_t |x_t|}{\sqrt{\frac{1}{n}\sum_t x_t^2}} < 8$$

ОЦІНЮВАННЯ

- Записи ESC-50 і Commons, кожен з окремого вихідного звуку; поділ 70/30 **за файлами** зі стратифікацією.
- Базовий рівень — zero-shot YAMNet; адаптація — YAMNet + LogisticRegression. Метрики: accuracy, F1-macro.
- Демонстрація: хвиля, мел-спектрограма, прогноз обох методів і топ-5 класів AudioSet.

ЧОМУ ТАК

- Поділ за файлами, а не за фрагментами: сусідні 0,96-секундні шматки одного запису майже однакові — інакше тест «підглядав» би в навчання.
- Середнє ловить сталий фон (дощ, гул двигуна), максимум — короткі яскраві події (наростання сирени).
- Клас «бензопила / мопед» навмисний: його звук схожий на Шахед, тож модель вчиться розрізняти саме цю пару.
- Zero-shot корисний як чесний базовий рівень: у AudioSet немає класу «Шахед», тож донавчання має це перевершити.
- Записів Шахедів у відкритому доступі мало (реактивних — одиниці), тому оцінка для них ненадійна — це треба назвати на доповіді як обмеження.

РОЗМОРОЖУВАННЯ YAMNET (ЕКСПЕРИМЕНТ)

YAMNet складається з 14 згорткових блоків (MobileNet v1). Звичайна адаптація їх не змінює. В експерименті верхні L блоків навчаються разом із лінійною головою (старт — з логістичної регресії, **LP-FT**).

Ризик: записів мало (≈ 116 у навчанні, Шахедів — одиниці), а розморожених ваг — від сотень тисяч до мільйонів. Модель легко **перенавчається**: запам'ятовує навчальні записи і гірше працює на нових. Якщо так сталося — це не збій, а ілюстрація, чому на малих даних шари заморожують.

Технічно YAMNet завантажується як збережена модель TensorFlow Hub; якщо її версія не дає рахувати градієнти, програма повідомить і залишить звичайну адаптацію.

9. Модуль 04 — Кластеризація інформаційних повідомлень

Готова модель: MiniLM + KMeans.

Навіщо це військовим

Черговий отримує потік коротких повідомлень без жодних позначок. Кластеризація сама розкладає їх на теми — логістика, погода, техніка, зв'язок, медицина, навчання — і показує, що зараз «гаряче». Ніхто не мусить заздалегідь розмічати повідомлення.

Простий приклад

Висипте на стіл 60 записок і попросіть розкласти їх на купки «за змістом», не кажучи, які купки бувають. Людина згрупує «ожеледиця» з «заморозками», а «пальне» з «доставкою». KMeans робить те саме з ембеддингами: схожі за змістом повідомлення опиняються поруч, і алгоритм ділить простір на k груп.

Як працювати

1. **Модель і дані** — 60 синтетичних повідомлень; приховані теми (6) використовуються лише для оцінки.
2. **Кластеризація** — оберіть k (за замовчуванням 6) → «Згрупувати». Карта PCA або t-SNE (наведіть на точку — побачите текст), автоматичні назви й ключові слова кластерів, найтиповіший текст.
3. **Оцінювання** — silhouette, ARI, NMI, «Чому так вийшло» (чи вдалий k, які кластери змішані); графіки вибору k: silhouette і метод «ліктя»; таблиця «кластер × тема».
4. **Демонстрація** — нове повідомлення з'являється зіркою на карті серед наявних.

Що показати на доповіді

Покажіть карту: точки однієї теми злипаються, хоча алгоритм не знав тем. Потім — графік «ліктя»: «k = 6 обрано не навмання».



Кластеризація: повідомлення на площині, колір — кластер



Вибір k: silhouette і метод «ліктя»

Теорія

ІДЕЯ

Навчання без учителя. Міток немає: 60 повідомлень перетворюються на ембеддинги MiniLM і групуються алгоритмом KMeans за смисловою близькістю. Це **використання готових ознак, а не донавчання** — ваги мережі не змінюються жодним чином. Назви кластерів програма складає автоматично з ключових слів.

МЕТОДИ І ФОРМУЛИ

KMeans шукає k центрів, що мінімізують суму квадратів відстаней до найближчого центру (алгоритм Ллойда, ініціалізація k-means++, 10 перезапусків):

$$\min_{C_1..C_k} \sum_{j=1}^{k} \sum_{x \in C_j} \|x - \mu_j\|^2$$

Для L2-нормалізованих ембеддингів евклідова відстань пов'язана з косинусною схожістю, тож KMeans групує за змістом:

$$\|u - v\|^2 = 2\left(1 - \cos(u, v)\right)$$

Silhouette точки i: a — середня відстань до своїх, b — до найближчого чужого кластера:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}} \in [-1;\, 1]$$

Метод «ліктя» — графік суми квадратів відстаней (inertia) для $k = 1..9$: де крива перестає різко падати, там розумне k.

ARI (скоригований індекс Ренда) порівнює кластери з прихованими темами з поправкою на випадковість; **NMI** — нормалізована взаємна інформація:

$$\mathrm{ARI} = \frac{RI - \mathbb{E}[RI]}{\max RI - \mathbb{E}[RI]}, \qquad \mathrm{NMI} = \frac{2\, I(U;V)}{H(U) + H(V)}$$

ВІЗУАЛІЗАЦІЯ

- **PCA** — проєкція 384-вимірних векторів на дві осі найбільшої дисперсії (власні вектори коваріаційної матриці). Лінійна, швидка; нові точки можна додавати.
- **t-SNE** — нелінійне вкладення, що зберігає локальних сусідів. Perplexity ≈ «кількість сусідів»; для 60 точок беремо 5–10.
- **Ключові слова й назва кластера** — c-TF-IDF: тексти кластера склеюються в один документ, беруться найвагоміші слова.

ЧОМУ ТАК

- Базовий рівень KMeans на TF-IDF групує за спільними словами; MiniLM — за змістом («ожеледиця» і «заморозки» поруч, хоча слів спільних немає).
- Вибір k обґрунтовують два графіки — silhouette і метод «ліктя», а не здогадка.
- Приховані еталонні теми не беруть участі в кластеризації — лише в оцінці ARI/NMI.

10. Модуль 05 — Міні-LLM для створення навчальних тестів

Готова модель: Qwen3-0.6B.

Навіщо це військовим

Викладач чи інструктор витрачає години на складання тестів. Мала мовна модель за навчальним текстом пропонує запитання з варіантами відповідей або твердження «правда / неправда»; викладач перевіряє, правит і експортує тест у Word чи Moodle. Модель працює локально — тексти нікуди не надсилаються.

Простий приклад

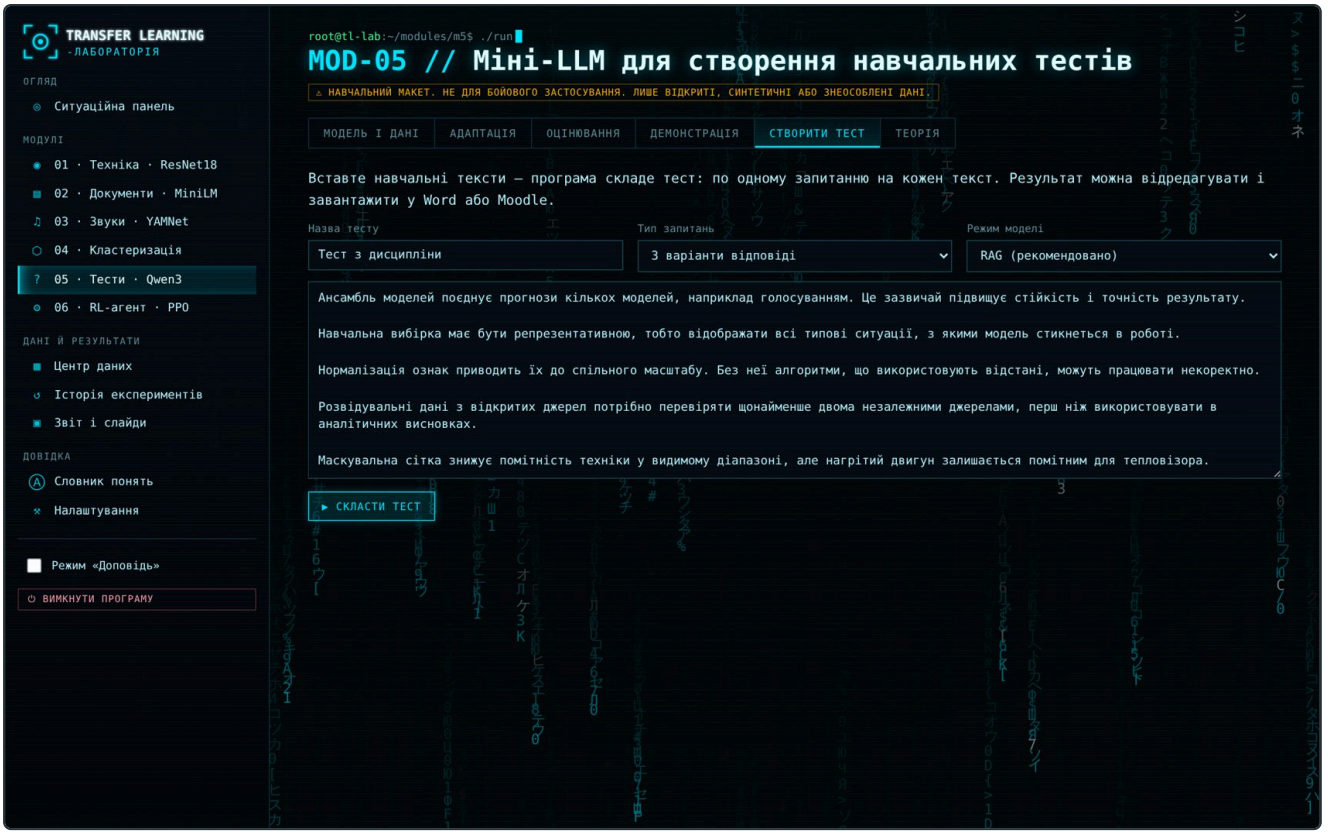
Новому писарю показують три зразки рапортів — і він пише четвертий у тому ж форматі (few-shot). Якщо дати йому саме ті зразки, що найближчі до теми, — результат кращий (RAG). А якщо кілька днів тренувати на зразках, він засвоїть формат назавжди (LoRA).

Як працювати

1. **Модель і дані** — 80 прикладів «текст → запитання»: 60 з трьома варіантами, 20 «правда / неправда».
2. **Адаптація** — позначте режими → «Оцінити режими» (на CPU 5–12 хв). LoRA-донавчання — тут, якщо є відеокарта, або в Google Colab (див. нижче).
3. **Оцінювання** — частка валідного JSON і правильних ключів за режимами, «Чому так вийшло», усі згенеровані запитання.
4. **Демонстрація** — текст + тип запитання → відповіді трьох режимів поруч.
5. **Створити тест** — вставте кілька абзаців → по запитанню на кожен → перевірте й виправте → «Word (.docx)» або «Moodle (GIFT)».

Що показати на доповіді

Згенеруйте запитання до одного абзацу трьома режимами й порівняйте. Скажіть: «Модель — помічник, а не викладач: тест завжди перевіряє людина».



«Створити тест»: тексти → запитання → експорт у Word і Moodle

LoRA-донавчання в Google Colab (якщо немає відеокарти)

1. Візьміть оригінальний `TL-Lab.zip` (або заархівуйте папку `TL-Lab` без `runtime` і `app\models`).
2. Відкрийте `app\notebooks\lora_colab.ipynb` на colab.research.google.com («Файл → Завантажити блокнот») → «Середовище виконання → Змінити тип → GPU T4».
3. Крок 1, варіант A: виконайте комірку і оберіть `TL-Lab.zip`; далі виконуйте комірки кроків 2–6 по черзі (≈ 15–20 хв).
4. Остання комірка завантажить `qwen3-lora.zip` — розпакуйте його в `TL-Lab\app\models\` (має вийти `models\qwen3-lora\adapter_config.json`).
5. Перезапустіть програму (`TL-Lab-Stop.bat`, потім ярлик) — у модулі 05 з'явиться режим LoRA.

У Colab потрапляють лише синтетичні навчальні приклади з архіву програми.

Теорія

ІДЕЯ

Невелика генеративна мовна модель за навчальним текстом складає тестове запитання у форматі JSON — з трьома варіантами відповіді або твердження «правда / неправда». Порівнюються чотири способи: **zero-shot** (лише інструкція), **few-shot** (3 приклади в запиті), **RAG** (3 найсхожіші приклади, знайдені пошуком), **LoRA** (донавчений адаптер). Тільки LoRA змінює ваги; few-shot і RAG — це навчання «в контексті» без зміни ваг.

АРХІТЕКТУРА І ВАГИ

- **Qwen3-0.6B** (Alibaba, 2025): декодерний трансформер, 28 шарів, ≈ 0,6 млрд параметрів, групова увага (16 голів запитів / 8 голів ключів-значень), контекст 32 768 токенів, багатомовна.

- Має «режим міркувань» — вимикаємо (`enable_thinking=False`), щоб модель одразу видавала JSON.
- Працює в **окремому процесі** програми: якщо забракне пам'яті, програма не закриється.
- **LoRA** додає до проєкцій уваги `q_proj` і `v_proj` кожного шару малі матриці рангу $r = 8$. Навчається ≈ **1,1 млн** параметрів (≈ 0,2 % моделі); основні ваги заморожені.

ФОРМУЛИ

Мовна модель передбачає наступний токен за попередніми:

$$P(x_1..x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

Під час донавчання втрата рахується **лише на токенах відповіді** (токени запиту замасковані):

$$\mathcal{L} = -\sum_{t \in \text{відповідь}} \log P(x_t \mid x_{<t})$$

LoRA (Hu et al., 2021): оновлення ваг подається як добуток двох вузьких матриць, B ініціалізується нулями, тож на старті модель не змінюється:

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d},\ B \in \mathbb{R}^{d \times r},\ r = 8,\ \alpha = 16$$

RAG: приклади для запиту обираються за косинусною схожістю ембеддингів MiniLM:

$$\text{top-}k = \arg\max_{i}\ \cos\big(e(\text{текст}),\ e(\text{приклад}_i)\big), \quad k = 3$$

Генерація — жадібна: на кожному кроці береться найімовірніший токен.

ОЦІНЮВАННЯ

- 80 синтетичних прикладів за темами курсу: 60 запитань із трьома варіантами і 20 тверджень «правда / неправда». Для тесту відкладено 9 (6 + 3), решта — для навчання LoRA і прикладів few-shot / RAG.
- Автоматичні метрики: частка **валідного JSON** і частка відповідей, де **ключ є серед варіантів**.
- Експертна оцінка 1–5: відповідність тексту, правильність ключа, якість дистракторів (неправильних варіантів).
- Розділ «Створити тест»: по одному запитанню на кожен абзац, ручне редагування, експорт у Word і Moodle (GIFT).

ЧОМУ ТАК

- 70 прикладів замало, щоб навчити модель новим знанням, але достатньо, щоб навчити **формату** — саме це й робить LoRA.
- Повне донавчання 0,6 млрд параметрів потребує десятків ГБ пам'яті; LoRA вміщується в безкоштовний GPU Colab.
- Модель 0,6B слабко володіє українською й може «галюцинувати» — тому тест завжди перевіряє викладач.

11. Модуль 06 — Донавчання RL-агента у зміненому середовищі

Готова модель: PPO CartPole і Acrobot (rl-zoo).

Навіщо це військовим

Система керування (дрон, стабілізатор, автопілот) навчена в одних умовах, а працювати їй доводиться в інших: вітер, інша вага, неточні датчики. Замість навчання з нуля готового агента **доучують** у нових умовах — швидко і з перевіркою, що він не забув старе. Модуль показує цей принцип на простих симуляторах.

Простий приклад

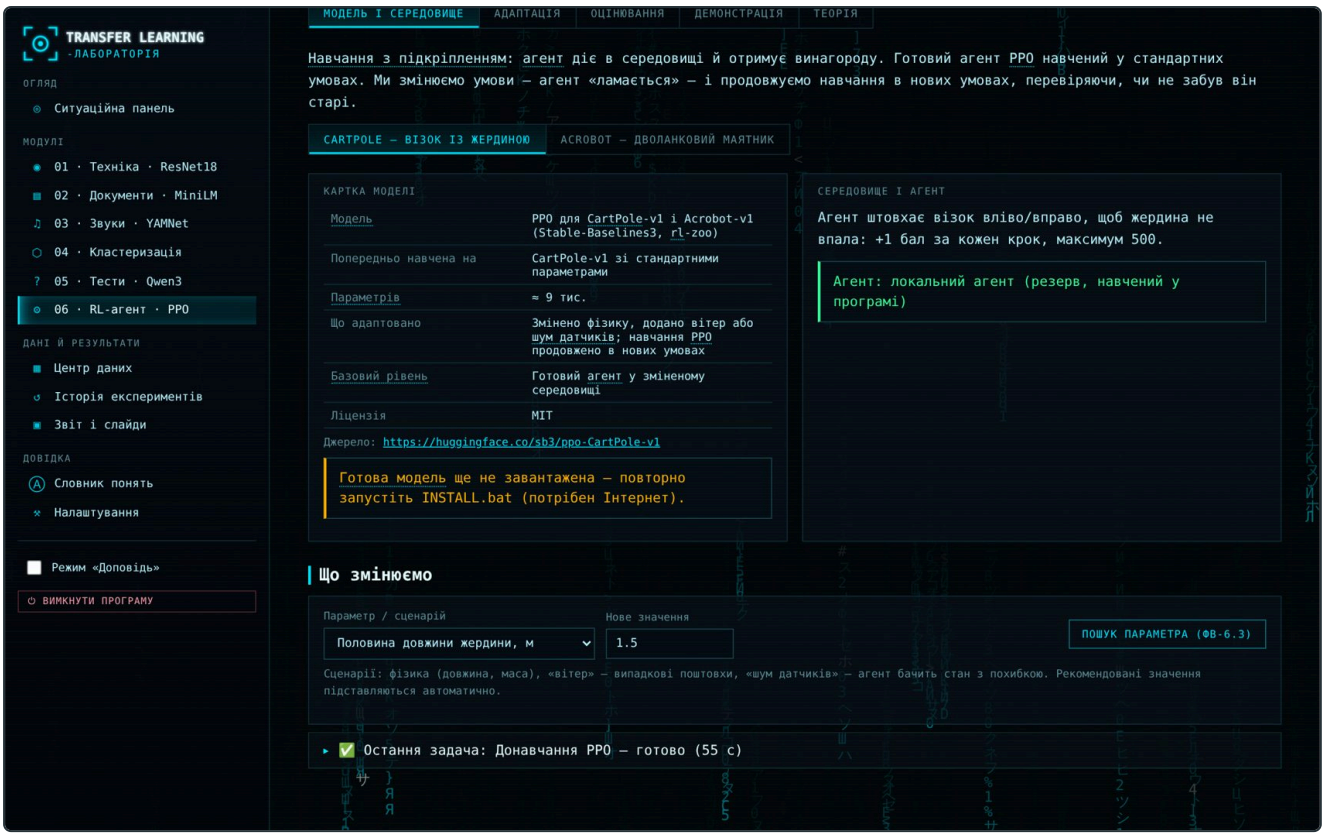
Водій, який роками їздив легковиком, пересідає на вантажівку: перші хвилини незграбно, але за кілька поїздок пристосовується — і не розучується водити легковик. Агент PPO так само: після зміни довжини жердини падає, після донавчання — знову тримає, а контроль у стандартних умовах перевіряє, чи не «забув» він старе.

Як працювати

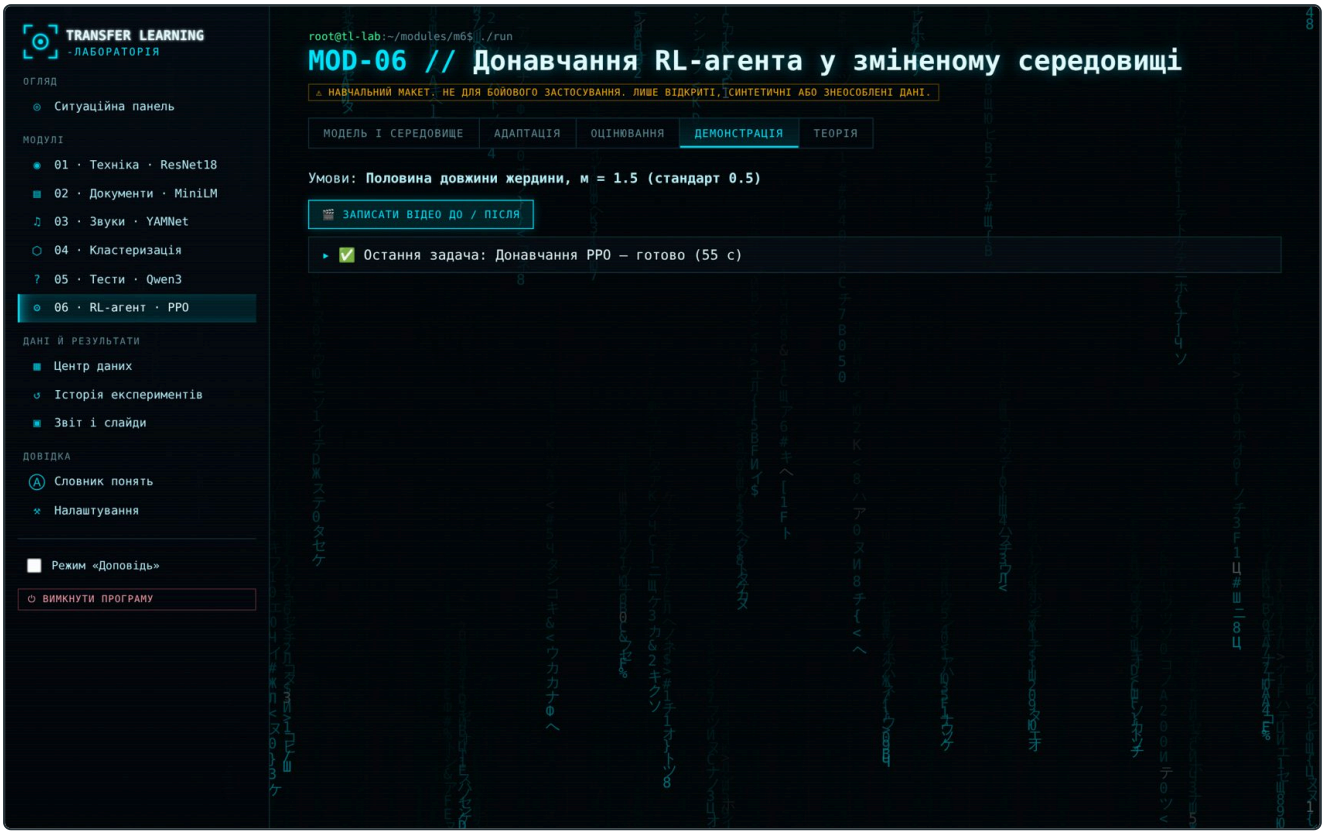
1. **Модель і середовище** — оберіть середовище (CartPole або Acrobot) і що змінюємо: фізику, вітер чи шум датчиків (рекомендоване значення підставляється). «Пошук параметра» показує, наскільки зміна шкодить агенту: потрібне падіння ≥ 30 % (нижче жовтої лінії).
2. **Адаптація** — кроки донавчання (50 000 ≈ 1–2 хв) → «Запустити донавчання»; у консолі — середня винагорода кожні 20 епізодів, після завершення — крива навчання.
3. **Порівняння (опція)** — галочка «Порівняти з частковим донавчанням»: той самий старт, але тіло мережі заморожене, навчається лише останній шар (≈ 200 ваг замість ≈ 9 000). Час — удвічі більше.
4. **Оцінювання** — табло «до → після» у зміненому середовищі, «Чому так вийшло», таблиця «повне / часткове» і контроль забування в стандартному середовищі.
5. **Демонстрація** — «Записати відео до / після»: ліворуч агент до донавчання, праворуч — після.

Що показати на доповіді

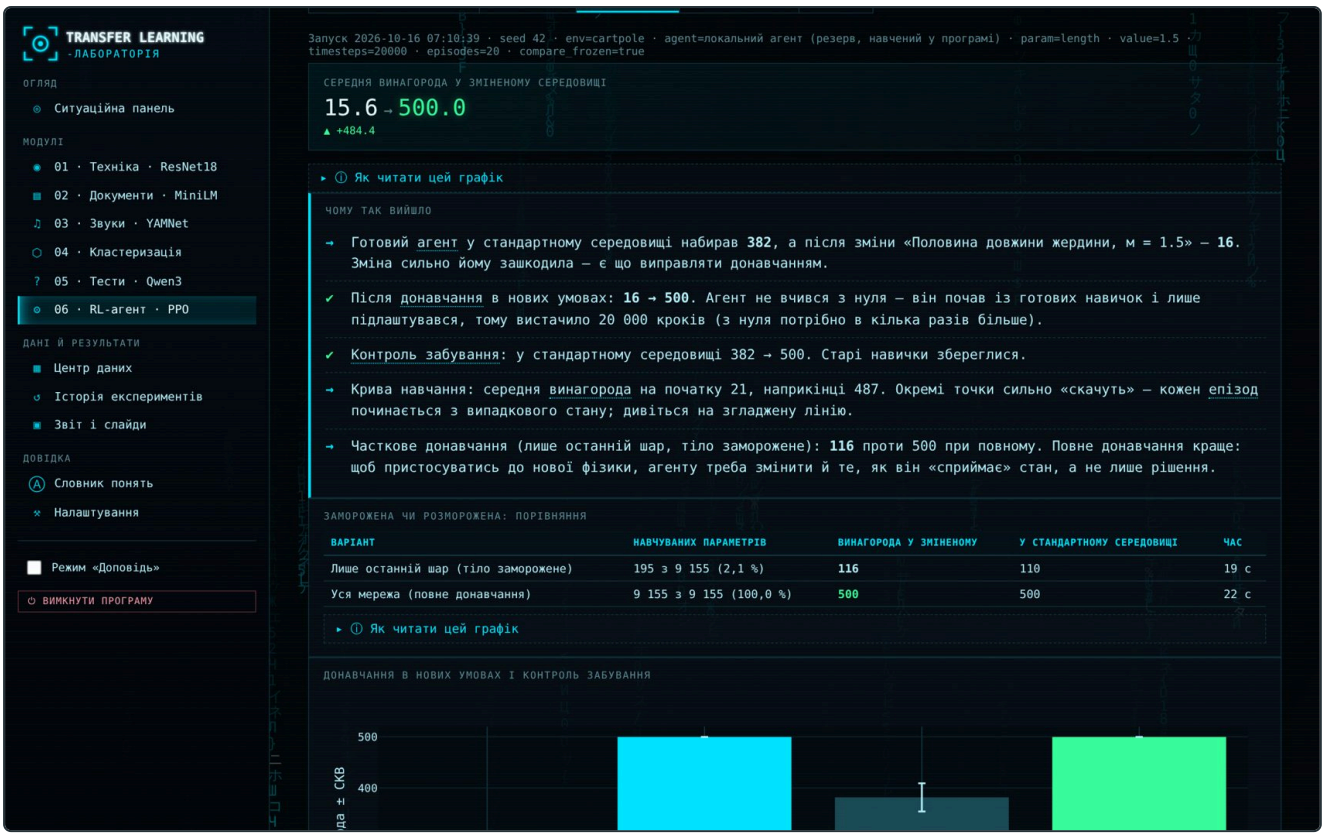
Найефектніше: CartPole, «Половина довжини жердини» = 1,5. До — жердина падає за ~16 кроків, після — тримається всі 500. Відео «до / після» говорить саме за себе.



«Пошук параметра»: падіння винагороди залежно від зміни



Відео: ліворуч — до донавчання (жердина падає), праворуч — після



Оцінювання: «Чому так вийшло» і порівняння повного та часткового донавчання

Теорія

ІДЕЯ

Навчання з підкріпленням. Агент діє в середовищі й отримує винагороду. Готовий агент PPO навчений у стандартних умовах. Ми **змінюємо умови** — фізику (довжина, маса), **вітер** (випадкові поштовхи) або **шум датчиків** (агент бачить стан з похибкою) — агент «ламається» — і **продовжуємо навчання** в нових умовах.

Два середовища: **CartPole** (утримати жердину на візку: +1 за крок, максимум 500) і **Acrobot** (розгойдати дволанковий маятник вище лінії: −1 за крок, ближче до 0 — краще).

СЕРЕДОВИЩЕ CARTPOLE-V1

Стан: $(x, \dot{x}, \theta, \dot{\theta})$ — положення й швидкість візка, кут і кутова швидкість жердини. Дія: поштовх вліво/вправо силою $F = \pm 10$ Н. Епізод завершується при $|\theta| > 12°$, $|x| > 2{,}4$ або після 500 кроків. Динаміка (Barto, Sutton, Anderson, 1983), l — **половина** довжини жердини:

$$\ddot{\theta} = \frac{g \sin\theta - \cos\theta \cdot \frac{F + m_p l \dot{\theta}^2 \sin\theta}{m_c + m_p}}{l\left(\frac{4}{3} - \frac{m_p \cos^2\theta}{m_c + m_p}\right)}, \qquad \ddot{x} = \frac{F + m_p l \dot{\theta}^2 \sin\theta}{m_c + m_p} - \frac{m_p l \ddot{\theta} \cos\theta}{m_c + m_p}$$

Інтегрування методом Ейлера з кроком 0,02 с. Похідні величини ($m_c + m_p$, $m_p l$) після зміни параметра перераховуються вручну.

Вітер: на кожному кроці до швидкостей додається випадковий поштовх $\xi \sim \mathcal{N}(0, \sigma^2)$. **Шум датчиків**: агент бачить $\tilde{s} = s + \eta$, $\eta \sim \mathcal{N}(0, \sigma^2)$, а фізика працює з точним s.

СЕРЕДОВИЩЕ ACROBOT-V1

Дві ланки, з'єднані шарніром; двигун лише в середньому шарнірі (дії: −1, 0, +1 Н·м). Мета — підняти кінець нижньої ланки вище лінії на висоті однієї ланки над віссю. Стан — синуси й косинуси двох кутів і дві кутові швидкості (6 чисел). Змінюємо масу нижньої ланки, довжину чи масу верхньої — маятник розгойдується інакше.

АЛГОРИТМ PPO

Proximal Policy Optimization (Schulman et al., 2017) обмежує, наскільки нова політика може відрізнятися від старої за одне оновлення:

$$r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)}, \qquad L^{CLIP} = \mathbb{E}_t\Big[\min\big(r_t\hat{A}_t,\ \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t\big)\Big],\ \varepsilon = 0{,}2$$

Перевага дії оцінюється методом GAE:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \qquad \hat{A}_t = \sum_{l\geq 0}(\gamma\lambda)^l\,\delta_{t+l}$$

Повна втрата: $L = -L^{CLIP} + c_v L^V - c_e H[\pi]$ (критик + бонус ентропії за дослідження). Політика й критик — дві мережі MLP 4 → 64 → 64 → (2 дії | 1 значення), разом ≈ **9 тис.** параметрів (CartPole).

ОЦІНЮВАННЯ

- Однакові **20 епізодів з фіксованими seed** до і після, детерміністичні дії — порівняння чесне, без шуму.
- Метрика — середня винагорода ± СКВ у зміненому середовищі.
- **Контроль забування**: той самий агент після донавчання перевіряється у стандартному середовищі.
- Перед донавчанням «Пошук параметра» показує, наскільки зміна шкодить агенту; потрібне падіння ≥ 30 %.
- Демонстрація — одне відео: ліворуч агент до донавчання, праворуч — після, з однаковим стартом.

ЧОМУ ТАК

- Перевірка показала: агент майже не чутливий до сили тяжіння (449 → 403 навіть за 40 м/с²), тож зміна g нічого не продемонструє. Довжина жердини ×3 обвалює винагороду до ~16 — є що виправляти.
- Донавчання з готових ваг (≈ 50 тис. кроків, 1–2 хв) у рази швидше за навчання з нуля — це і є перевага повторного використання моделі в RL.
- Симулятори спрощені; висновки — про принцип адаптації, а не про конкретну бойову систему.

ПОВНЕ ЧИ ЧАСТКОВЕ ДОНАВЧАННЯ (ОПЦІЯ)

Мережа політики PPO: «тіло» (2 шари по 64 нейрони — окремо для вибору дії і для оцінки стану) перетворює стан середовища на внутрішні ознаки, а останній шар обирає дію (і окремий — оцінює стан).

- **Повне донавчання** (за замовчуванням): змінюються всі ≈ 9 тис. ваг.
- **Часткове**: тіло заморожене, навчається лише останній шар (≈ 200 ваг).

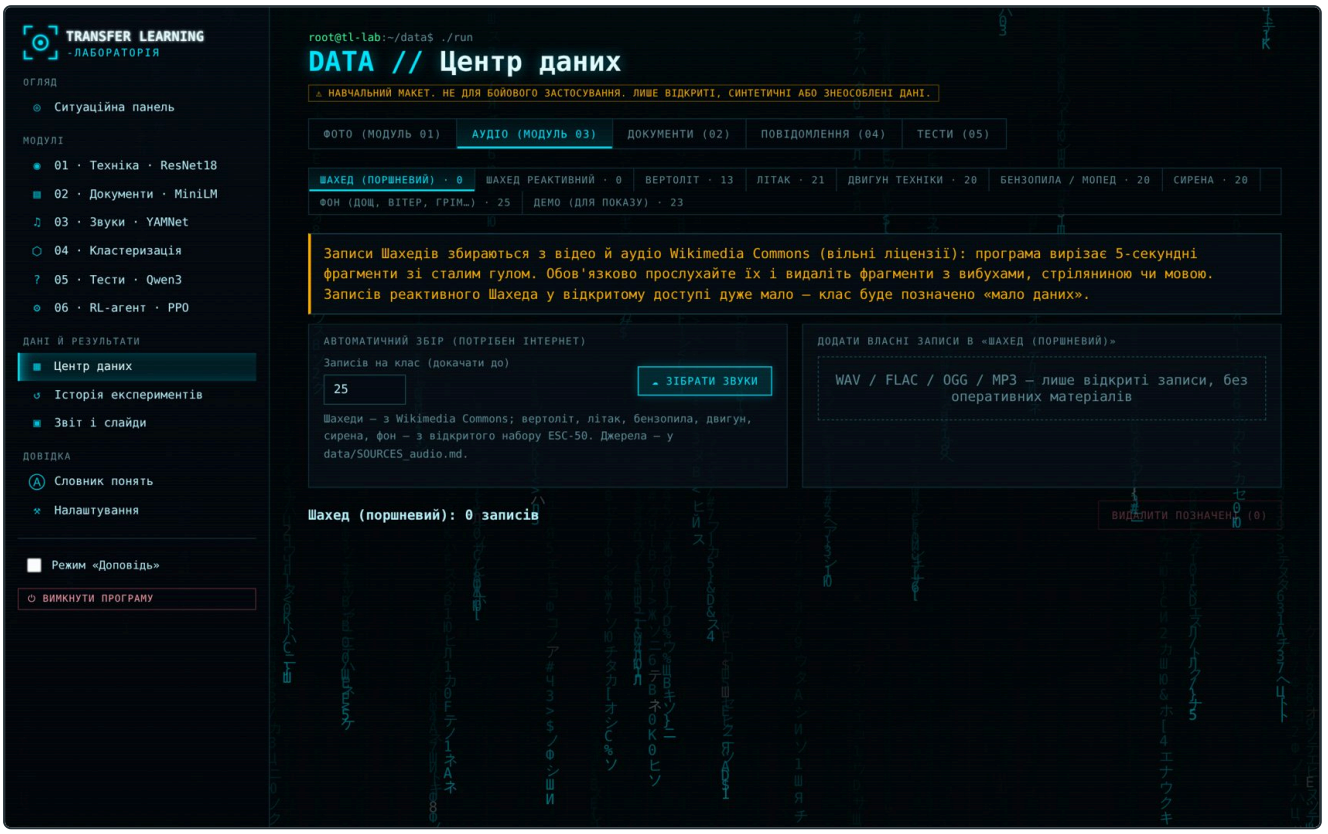
Якщо зміна середовища «перекроює» фізику (довга жердина), агенту часто треба по-новому сприймати стан — повне донавчання виграє. Якщо зміна невелика (шум), вистачає останнього шару — і менше ризику **забути** старі навички.

12. Центр даних

Усі дані модулів — в одному місці, без роботи з папками.

- **Фото (модуль 01)** — галерея всіх фото класу (клік — позначити, подвійний клік — збільшити), «Видалити позначені», перетягування власних фото, **«Зібрати фото для всіх класів»** з Wikimedia Commons (вказуєте, до скількох докачати; наявні не чіпаються; автори й ліцензії — у `data\SOURCES_images.md`).
- **Дублікати** — «Знайти дублікати» показує групи майже однакових фото поруч (зелена рамка — залишиться, червона — буде видалено); **«Знайти й видалити дублікати»** робить це одним кліком. Дублікати в навчальній і тестовій вибірках завищують точність, тому їх варто прибрати перед навчанням.
- **Аудіо (модуль 03)** — записи за класами з прослуховуванням, видалення, додавання своїх; **«Зібрати звуки»**: Шахеди — з відео й аудіо Wikimedia Commons (вирізаються 5-секундні фрагменти зі сталим гулом), інші класи — з ESC-50.
- **Документи, Повідомлення, Тести** — редагування прямо в таблиці (клік по тексту), додавання й видалення рядків, «Зберегти». Документи можна **імпортувати** з `.docx` / `.txt` (кожен файл — один документ обраного типу).

> **Безпека даних.** Після автоматичного збору перегляньте фото й прослухайте звуки. Видаляйте знімки реальних позицій чи техніки ЗСУ з прив'язкою до місцевості, фрагменти з вибухами, стрілянною чи мовою. Не завантажуйте справжні службові документи й матеріали реальної оперативної обстановки — лише навчальні, вигадані або знеособлені.



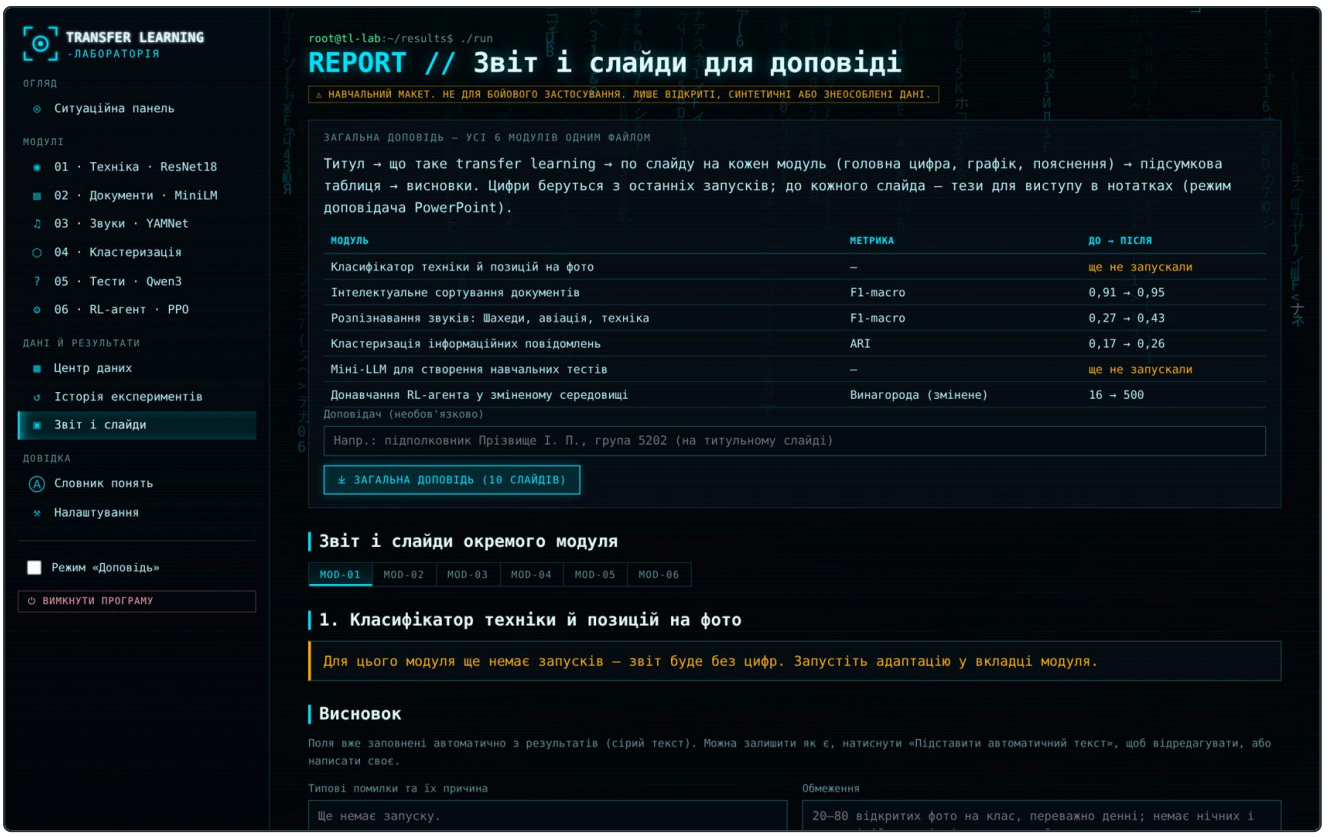
Центр даних → Аудіо: збір звуків і застереження щодо записів Шахедів

13. Історія, звіт і слайди

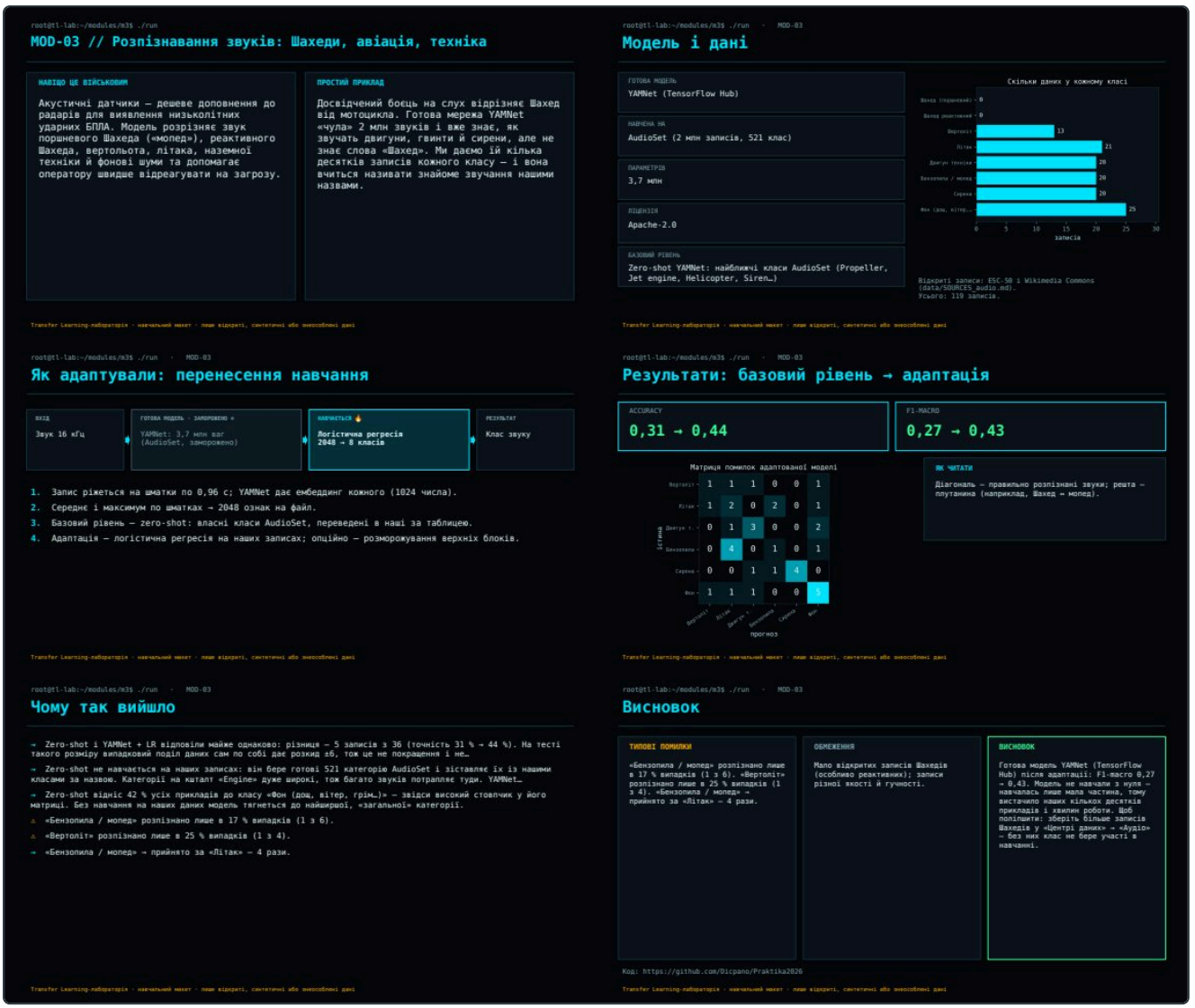
- **Історія експериментів** — усі запуски всіх модулів: метрика «база → адаптація», різниця, параметри; графік динаміки для обраного модуля; «Експорт CSV»; видалення невдалих запусків.
- **Загальна доповідь (10 слайдів)** — одна кнопка вгорі сторінки «Звіт і слайди»: титул → що таке transfer learning (схема) → по слайду на кожен із 6 модулів (головна цифра «до → після», графік, 2–3 пункти «Чому так вийшло») → підсумкова таблиця → висновки. Поле «Доповідач» потрапляє на титульний слайд.
- **Слайди модуля (6 шт.)** — 1) задача і навіщо це військовим; 2) модель і дані (діаграма кількості даних);
  3. як адаптували — схема «вхід → заморожена модель → що навчили → результат»; 4) результати: табло, графік, порівняння «заморожена / розморожена»; 5) чому так вийшло; 6) типові помилки, обмеження, висновок.
- **Нічого не треба писати вручну:** «Помилки», «Обмеження» і «Висновок» програма складає сама з результатів (сірий текст у полях). «Підставити автоматичний текст» — щоб відредагувати; або напишіть своє.
- **Тези для виступу** — у нотатках до кожного слайда: у PowerPoint «Показ слайдів → Режим доповідача» або «Вигляд → Нотатки».
- **Звіт (.md)** — повний текст: навіщо, модель і дані, як адаптували, результати, «Чому так вийшло», висновок.

Чек-лист перед захистом

- [ ] Програма запускається з ярлика **без Інтернету** (перевірте, вимкнувши Wi-Fi).
- [ ] Адаптацію свого модуля запущено заздалегідь; результат є в «Історії».
- [ ] Загальну доповідь і слайди свого модуля завантажено; вони відкриваються в PowerPoint, тези в нотатках прочитано.
- [ ] Підготовлено демо-приклади (фото / текст / звук), на яких модель працює впевнено, і один приклад помилки.
- [ ] На проєкторі фон вимкнено або приглушено, увімкнено режим «Доповідь».
- [ ] Можете пояснити одну формулу свого модуля і кілька термінів зі словника.



Звіт і слайди: загальна доповідь, слайди модуля, автоматичний висновок



Шість слайдів модуля (приклад модуля 03 на тестових даних)

14. Словник понять

У програмі — сторінка «Словник понять» з пошуком; тут — повний перелік (81 понять). У дужках — модулі, де поняття трапляється.

Основи машинного навчання

- **Fine-tuning** (01, 05, 06) — Донавчання: продовження навчання готової моделі на нових даних. Може змінювати всі ваги або лише частину (останні шари, адаптер LoRA). Що менше даних, то менше ваг варто відкривати для змін. *Приклад:* У модулі 06 агент PPO донавчається в середовищі з довшою жердиною.
- **Learning rate** (01, 05, 06) — Швидкість навчання: наскільки сильно змінюються ваги за один крок. Завеликий крок — навчання «стрибає» і не сходиться, замалий — навчання дуже повільне. Типові значення 0,0001–0,003. *Приклад:* lr = 0,001 у модулі 01 — стандартне значення для оптимізатора Adam.
- **Softmax** (01, 02, 03) — Функція, що перетворює оцінки моделі на ймовірності, які в сумі дають 100 %. Модель видає для кожного класу число (логіт). Softmax підносить e в степінь кожного і ділить на суму — виходять ймовірності. *Приклад:* БПЛА 82 %, вантажівка 12 %, інше 6 % — це вихід softmax.
- **Transfer learning** (01, 02, 03, 05, 06) — Перенесення знань: готову модель пристосовують до нової задачі, донавчаючи лише невелику частину. Ранні шари готової мережі вже вміють виділяти корисні ознаки. Їх заморожують, а навчають тільки нову «голову» або невеликий адаптер. Тому вистачає десятків прикладів замість мільйонів. *Приклад:* Як досвідчений водій вантажівки швидко пересідає на БТР — базові навички вже є, вчить лише відмінності.
- **Ваги** (01, 02, 03, 05, 06) — Числа всередині моделі, які змінюються під час навчання і зберігають набуті знання. У ResNet18 їх 11,7 млн, у Qwen3 — 0,6 млрд. Навчання — це пошук таких ваг, за яких помилки найменші. *Приклад:* Заморозити ваги — заборонити їм змінюватися під час навчання.
- **Градієнтний спуск** (01, 05, 06) — Спосіб навчання: ваги змінюють у напрямку, де помилка зменшується найшвидше. Градієнт показує, як зміниться втрата при малій зміні кожної ваги. Оптимізатор (Adam, SGD) робить крок у протилежному напрямку. *Приклад:* Як спуск з гори в тумані: щоразу крок туди, де схил іде вниз.
- **Епоха** (01, 05) — Один повний прохід моделі по всіх навчальних прикладах. Зазвичай навчають кілька епох, бо за один прохід модель не встигає налаштуватися. Забагато епох — ризик перенавчання. *Приклад:* 10 епох у модулі 01 = модель 10 разів побачила кожне навчальне фото.
- **Класифікація** (01, 02, 03) — Задача віднести об'єкт до одного з відомих класів. Потрібні приклади з мітками («це рапорт», «це наказ»). Модулі 01–03 — класифікація фото, документів і звуків. *Приклад:* Документ → «Заявка на забезпечення».
- **Машинне навчання** (01, 02, 03, 04, 05, 06) — Підхід, коли модель вчиться на прикладах, а не за заздалегідь написаними правилами. Модель має параметри (ваги), які автоматично підлаштовуються так, щоб відповіді на навчальних прикладах були правильними. Далі її перевіряють на прикладах, яких вона не бачила. *Приклад:* Показали 30 фото БПЛА і 30 фото вантажівок — модель навчилася їх розрізняти.
- **Модель** (01, 02, 03, 04, 05, 06) — Навчена програма, яка за вхідними даними видає відповідь (клас, число, текст, дію). Модель = архітектура (як влаштована) + ваги (чого навчилася).

Готові моделі зберігаються у файлах і завантажуються з репозиторіїв. *Приклад:* Файл models/m1_resnet18.pt — це навчена модель модуля 01.

- **Нейронна мережа** (01, 02, 03, 05, 06) — Модель із шарів простих обчислювальних елементів (нейронів), з'єднаних вагами. Кожен нейрон множить входи на ваги, додає їх і пропускає через функцію активації. Шари йдуть один за одним: ранні виділяють прості ознаки, глибші — складні. *Приклад:* ResNet18 у модулі 01 — нейронна мережа з 18 шарів з вагами.
- **Попередньо навчена модель** (01, 02, 03, 04, 05, 06) — Модель, яку вже навчили на великому наборі даних і яку можна використати повторно. Навчання з нуля потребує мільйонів прикладів і тижнів обчислень. Готову модель беруть з відкритого репозиторію (PyTorch, Hugging Face, TensorFlow Hub) і пристосовують до своєї задачі. *Приклад:* ResNet18 навчена на ImageNet, YAMNet — на AudioSet, MiniLM — на парафразах 50+ мов.
- **Регуляризація** (02, 03) — Штраф за надто великі ваги, щоб модель не запам'ятовувала дані, а узагальнювала. Параметр C у логістичній регресії: що менше C, то сильніша регуляризація і простіша модель. *Приклад:* C = 4 у модулі 02 — помірна регуляризація для 156 текстів.
- **Функція втрат** (01, 05, 06) — Число, яке показує, наскільки модель помиляється; навчання намагається його зменшити. Для класифікації — перехресна ентропія: що менша ймовірність правильної відповіді, то більший штраф. Крива втрат має спадати. *Приклад:* Якщо модель дає 90 % на правильний клас, штраф малий; якщо 10 % — великий.
- **Штучний інтелект** (01, 02, 03, 04, 05, 06) — Програми, які виконують завдання, що зазвичай потребують людського мислення. Сюди належать розпізнавання зображень і мови, переклад, планування дій. Машинне навчання — найпоширеніший сучасний спосіб створювати такі програми: їх не програмують правилами, а навчають на прикладах. *Приклад:* Оператор не описує правилами, як виглядає танк, — модель сама вчиться цьому на фото.

Дані й оцінювання

- **Accuracy** (01, 02, 03) — Точність: частка правильних відповідей серед усіх. Проста, але оманлива метрика при нерівних класах: якщо 90 % записів — фон, модель «завжди фон» матиме 90 %. *Приклад:* 45 правильних з 50 → accuracy 0,9.
- **F1-macro** (01, 02, 03) — Середнє F1 по всіх класах; F1 — гармонійне середнє precision і recall. Macro означає, що кожен клас важить однаково, тож рідкісний клас не «губиться». Головна метрика модулів 01–03. *Приклад:* F1-macro 0,92 — модель добре розрізняє всі класи, а не лише найчастіший.
- **Precision і Recall** (01, 02, 03) — Precision — скільки знайденого справді правильне; Recall — скільки правильного вдалося знайти. Для виявлення загроз важливий recall (не пропустити), для звітів — precision (не підняти хибну тривогу). *Приклад:* З 10 сигналів «Шахед» 8 справжніх — precision 0,8; знайдено 8 з 16 реальних — recall 0,5.
- **Seed** (01, 02, 03, 04, 05, 06) — Початкове значення генератора випадкових чисел; однаковий seed — однаковий результат. Фіксований seed робить експерименти відтворюваними: повторний запуск дасть ті самі цифри. У програмі seed = 42. *Приклад:* Однакові 20 seed епізодів «до» і «після» у модулі 06 — чесне порівняння.
- **Базовий рівень** (01, 02, 03, 04, 05, 06) — Простий метод, з яким порівнюють адаптовану модель, щоб довести користь адаптації. Без базового рівня не зрозуміло, чи 85 % — це добре. У кожному модулі є свій базовий рівень. *Приклад:* Модуль 02: TF-IDF (збіг слів) проти MiniLM (зміст).
- **Витік даних** (01, 03) — Ситуація, коли інформація з тесту потрапила в навчання і завищила оцінку. Типові причини: дублікати фото в обох вибірках, сусідні фрагменти одного запису в

train і test. Тому в модулі 03 поділ — за файлами. *Приклад:* Одне й те саме фото танка в навчанні й тесті — модель «впізнає», а не розпізнає.

- **Дублікати** (01) — Майже однакові фото в наборі, що завищують точність і спотворюють навчання. Програма шукає їх простим «відбитком» aHash: фото зменшують до 8×8 і порівнюють візерунок світлих і темних клітинок. *Приклад:* Кнопка «Знайти й видалити дублікати» в «Центрі даних» залишає по одному фото з кожної групи.
- **Знеособлення** (02, 04) — Видалення з даних того, що дозволяє встановити особу чи місце. ПІБ, номери, координати, позивні замінюють на умовні. Обов'язково перед використанням даних поза захищеним контуром. *Приклад:* «майор Іваненко, позиція біля Н-ська» → «майор [ПІБ], район навчання «Північ»».
- **Крос-валідація** (02) — Оцінка, коли дані по черзі ділять на k частин і кожна з них буває тестовою. Кожен приклад отримує прогноз від моделі, яка його не бачила. На малих наборах оцінка стабільніша, ніж на одному поділі. *Приклад:* 5-кратна: 156 текстів → 5 частин по ~31, п'ять навчань, усі тексти перевірено.
- **Матриця помилок** (01, 02, 03) — Таблиця «істина × прогноз»: на діагоналі правильні відповіді, поза нею — помилки. Показує, які саме класи модель плутає між собою, — це підказує, яких даних бракує. *Приклад:* Клітинка «бронетехніка → артилерія = 3»: три танки модель назвала артилерією.
- **Навчальна і тестова вибірки** (01, 02, 03, 05) — Дані ділять на частину для навчання і частину для чесної перевірки. Тестові приклади модель не бачить під час навчання. Тільки так оцінка показує, як модель працюватиме на нових даних. *Приклад:* 70 % фото — для навчання, 30 % — для тесту.
- **Перенавчання** (01, 02, 03, 05) — Модель запам'ятала навчальні приклади, але погано працює на нових. Ознака — висока точність на навчальних даних і низька на тестових. Ліки: більше даних, аугментація, регуляризація, менше ваг, що навчаються. *Приклад:* Модель вивчила, що на всіх фото БПЛА — небо, і тепер «бачить» БПЛА в кожній хмарі.
- **Поріг впевненості** (01) — Мінімальна ймовірність, за якої модель дає відповідь; нижче — «Невизначено». Краще чесне «не знаю», ніж упевнена помилка. Поріг змінюється в «Налаштуваннях». *Приклад:* Поріг 60 %: «БПЛА 55 %» → програма скаже «Невизначено».
- **Синтетичні дані** (02, 04, 05) — Штучно створені приклади, що імітують справжні, але не містять реальної інформації. Дозволяють навчати й показувати моделі без ризику розголошення. Недолік — шаблонність: реальні тексти різноманітніші. *Приклад:* Усі документи модуля 02 вигадані: «навчальна рота», «умовний противник».
- **Стратифікація** (01, 02, 03) — Поділ даних так, щоб частки класів у навчальній і тестовій вибірках були однаковими. Без неї в тест може випадково не потрапити жоден приклад рідкісного класу. *Приклад:* Якщо 20 % фото — артилерія, то й у тесті ≈ 20 %.
- **Шум випадковості** (01, 02, 03, 05) — Розкид результату, який дає сам випадковий поділ даних, а не різниця між методами. На тесті з n прикладів точність «гуляє» приблизно на $\pm 2\cdot\sqrt{p(1-p)\cdot n}$ прикладів. Різниця, менша за це, — нічия. Програма враховує це в блоці «Чому так вийшло». *Приклад:* Тест 144 фото, точність 90 %: розкид ≈ ±7 фото, тож різниця в 1 фото — випадковість.

Зображення

- **Grad-CAM** (01) — Теплова карта, що показує, на які ділянки фото спиралася мережа під час рішення. Беруться градієнти відповіді за картами останнього згорткового шару. Червоне — важливе для рішення, синє — ні. *Приклад:* Якщо «гаряче» небо, а не дрон, — модель навчилася на фоні; потрібні інші фото.

- **ImageNet** (01) — Великий набір фото (1,28 млн, 1000 класів), на якому навчають мережі для зображень. Серед класів є танки, вантажівки, літаки, гармати — тому ознаки ImageNet добре переносяться на військову техніку. *Приклад:* ResNet18 у модулі 01 «бачила» ImageNet.
- **LP-FT** (02, 03) — Linear Probing → Fine-Tuning: спершу навчити лише останній шар, потім з нього стартувати розморожування. Якщо розморозити шари з випадковою «головою», перші кроки псують готові ознаки великими помилками. Старт голови з уже навченої логістичної регресії робить донавчання м'яким: розморожена модель починає з рівня замороженої. *Приклад:* У модулях 02 і 03 голова розмороженої моделі ініціалізується вагами логістичної регресії.
- **ResNet18** (01) — Згорткова мережа з 18 шарів і «залишковими з'єднаннями», навчена на ImageNet. Залишкове з'єднання додає вхід блоку до його виходу — завдяки цьому глибокі мережі добре навчаються. 11,7 млн ваг. *Приклад:* У модулі 01 — основа класифікатора техніки.
- **Аугментація** (01) — Штучне розширення набору: змінені копії фото (обрізання, віддзеркалення, яскравість). Кожна епоха бачить трохи інші варіанти тих самих знімків, тож модель вчиться суті, а не конкретному кадру. *Приклад:* Одне фото танка → 10 варіантів: ближче, дзеркально, темніше, під іншим кутом.
- **Заморожування шарів** (01, 02, 03, 06) — Заборона змінювати ваги частини мережі під час навчання. Заморожені шари працюють як готовий «екстрактор ознак». Це зменшує кількість ваг, що навчаються, прискорює навчання і зменшує ризик перенавчання на малих даних. *Приклад:* У модулі 01 заморожено 11,7 млн ваг, навчається лише 3 078 ваг нового шару (6 класів).
- **Згорткова нейронна мережа** (01) — Мережа для зображень, що «ковзає» по фото маленькими фільтрами й шукає візерунки. Ранні шари знаходять краї й плями кольору, середні — текстури й форми, глибокі — частини об'єктів (гусениці, гвинти, кабіну). *Приклад:* Фільтр 3×3 реагує на вертикальну межу — наприклад, на край ствола.
- **Маскування** (01) — Зменшення помітності техніки й позицій: сітки, природні матеріали, дисципліна світла й звуку. Для моделі замаскована позиція — складний клас: контури розмиті, кольори як у фону. Це хороший тест можливостей і меж моделі. *Приклад:* Клас «Замасковані позиції» в модулі 01 — лише фото з навчань і музеїв.
- **Розморожування шарів** (01, 02, 03, 06) — Дозвіл змінювати ваги кількох верхніх шарів готової мережі під час донавчання. Нижні шари бачать загальне (краї, звуки, слова) і корисні для будь-якого завдання — їх лишають замороженими. Верхні шари складають з цього специфічні ознаки — їх можна «підкрутити» під наші дані. Розморожування дає приріст, коли даних багато або вони дуже відрізняються від тих, на яких модель навчали; на малих наборах — ризик перенавчання. *Приклад:* Модуль 02: розморожено 2 з 12 шарів MiniLM — навчається ≈ 3,6 млн ваг замість 2 310.

Тексти

- **MiniLM** (02, 04, 05) — Компактна багатомовна мовна модель, що перетворює речення на ембеддинги (384 числа). paraphrase-multilingual-MiniLM-L12-v2: 12 шарів трансформера, 118 млн ваг, 50+ мов включно з українською. Навчена розпізнавати парафрази. *Приклад:* Модулі 02 і 04 використовують MiniLM, не змінюючи її ваг.
- **TF-IDF** (02, 04) — Простий спосіб описати текст вагами слів: часте в документі, рідкісне в інших — важливе. Не розуміє змісту і синонімів, бачить лише збіг слів. Тому слугує базовим рівнем у модулях 02 і 04. *Приклад:* Слово «прошу» важливе для рапорту, а «навчальний» трапляється всюди — його вага мала.

- **Ембеддинг** (02, 03, 04, 05) — Числовий вектор, що описує зміст тексту (або звуку): схожі за змістом — поруч. Модель перетворює текст на, наприклад, 384 числа. Тексти «станом на 08:00» і «за даними на ранок» отримують близькі вектори, хоча слова різні. *Приклад:* Як координати на карті змісту: донесення поруч з донесеннями, заявки — з заявками.
- **Косинусна схожість** (02, 04, 05) — Міра схожості двох векторів через кут між ними: 1 — однаковий зміст, 0 — нічого спільного. Використовується для пошуку найближчих прикладів (пояснення рішення в модулі 02, RAG у модулі 05). *Приклад:* Схожість 0,85 між двома заявками на пальне — вони майже про одне.
- **Логістична регресія** (02, 03) — Простий лінійний класифікатор, що оцінює ймовірність належності до кожного класу. Множить ознаки на ваги, додає зсув і застосовує softmax. Швидко навчається, мало перенавчається і добре працює поверх готових ембеддингів. *Приклад:* У модулях 02 і 03 логістична регресія вчиться на ембеддингах MiniLM і YAMNet.
- **Трансформер** (02, 04, 05) — Архітектура мовних моделей, де кожне слово «зважає» на всі інші слова тексту (механізм уваги). Саме трансформери стоять за сучасними мовними моделями: MiniLM, Qwen, GPT. Увага дозволяє враховувати контекст. *Приклад:* У фразі «ключ від складу» модель розуміє, що «ключ» — предмет, а не відповідь тесту.

Звук

- **AudioSet** (03) — Набір Google з ≈ 2 млн 10-секундних фрагментів YouTube, розмічених 521 класом звуків. Саме на ньому навчена YAMNet. Класу «Шахед» у ньому немає — тому потрібне донавчання на власних записах. *Приклад:* Найближчі класи AudioSet до Шахеда — Propeller, Motorcycle, Light engine.
- **ESC-50** (03) — Відкритий набір із 2000 п'ятисекундних звуків довкілля (50 класів), ліцензія CC BY-NC. Звідти взято записи вертольота, літака, бензопили, двигуна, сирени, дощу, вітру, грози. *Приклад:* Кожен запис узято з окремого вихідного звуку Freesound — щоб не було витоку даних.
- **YAMNet** (03) — Аудіомережа Google, навчена розпізнавати 521 клас звуків на наборі AudioSet. Для кожних 0,96 с звуку видає ймовірності 521 класу та ембеддинг з 1024 чисел. Працює в окремому процесі програми (TensorFlow). *Приклад:* YAMNet «знає» класи Helicopter, Jet engine, Siren, Chainsaw.
- **Zero-shot** (03, 05) — Використання готової моделі для нової задачі без жодного навчального прикладу. У модулі 03 — готові класи AudioSet; у модулі 05 — лише інструкція без прикладів. Чесний базовий рівень. *Приклад:* Без навчання YAMNet каже «Motorcycle» на звук Шахеда — схоже, але не те.
- **Акустичне виявлення** (03) — Виявлення об'єктів за звуком: мікрофони + модель, що впізнає характерний шум. Дешевий спосіб доповнити радари для виявлення низьколітних БПЛА. Обмеження: вітер, дощ, міський шум, дальність. *Приклад:* Поршневий Шахед має характерний «мопедний» звук двотактного двигуна.
- **Мало даних** (01, 03) — Позначка класу, для якого зібрано замало прикладів; оцінка для нього ненадійна. Класи з менш ніж 4 записами (5 фото) не беруть участі в навчанні, з менш ніж 10 — позначаються. Рішення — додати відкриті записи. *Приклад:* Реактивного Шахеда у відкритому доступі одиниці записів.
- **Мел-шкала** (03) — Шкала частот, що відповідає сприйняттю людиною: низькі частоти детальніше, високі — грубіше. YAMNet використовує 64 мел-смуги від 125 до 7500 Гц. *Приклад:* Різницю 100 і 200 Гц ми чуємо добре, а 7100 і 7200 Гц — майже ні.
- **Спектрограма** (03) — Зображення звуку: по горизонталі час, по вертикалі частота, колір — гучність. Звук перетворюють на «картинку», і далі з ним працюють методи розпізнавання

зображень. *Приклад:* Гул Шахеда — рівна горизонтальна смуга; вибух — короткий яскравий вертикальний спалах.

Кластеризація

- **ARI і NMI** (04) — Метрики збігу кластерів з еталонними темами: 1 — повний збіг, 0 — як випадково. ARI рахує пари об'єктів, що потрапили разом; NMI — скільки інформації про теми несуть кластери. Еталон використовується лише для оцінки. *Приклад:* ARI 0,8 — кластери майже збігаються з прихованими темами.
- **c-TF-IDF** (04) — Пошук ключових слів кластера: тексти кластера склеюють і беруть найважливіші слова. З цих слів програма автоматично складає назву кластера. *Приклад:* Кластер «пальне · склад · доставлено» — очевидно, логістика.
- **KMeans** (04) — Алгоритм, що ділить точки на k груп навколо k центрів. Повторює два кроки: кожну точку — до найближчого центру; кожен центр — у середину своїх точок. Зупиняється, коли нічого не змінюється. *Приклад:* k = 6: шість «центрів тяжіння» тем у потоці повідомлень.
- **PCA** (04) — Метод головних компонент: проєкція багатовимірних даних на 2–3 осі з найбільшим розкидом. Лінійний і швидкий. Дозволяє показати 384-вимірні ембеддинги на площині. *Приклад:* Кожна точка на графіку модуля 04 — повідомлення після PCA.
- **Silhouette** (04) — Оцінка якості кластерів від −1 до 1: наскільки точки близькі до своїх і далекі від чужих. Близько 1 — чіткі групи, близько 0 — групи перетинаються, від'ємне — точка, ймовірно, не в тому кластері. *Приклад:* Графік silhouette для k = 2…9 показує, яке k найкраще.
- **t-SNE** (04) — Нелінійна візуалізація, що розміщує схожі об'єкти поруч на площині. Добре показує локальні групи, але відстані між далекими групами не варто тлумачити буквально. Параметр perplexity ≈ кількість сусідів. *Приклад:* Для 60 точок perplexity 5–10.
- **Кластеризація** (04) — Групування об'єктів за схожістю без заздалегідь відомих міток. На відміну від класифікації, ніхто не каже моделі, які групи існують, — вона знаходить їх сама. *Приклад:* Потік із 60 повідомлень сам розпадається на «логістику», «погоду», «зв'язок»…
- **Метод ліктя** (04) — Спосіб обрати k: шукають «злам» на кривій суми відстаней до центрів. Зі зростанням k сума завжди падає, але після правильного k — повільно. Злам схожий на лікоть зігнутої руки. *Приклад:* Крива різко падає до k = 6, далі майже пласка → k = 6.
- **Навчання без учителя** (04) — Навчання на даних без міток: модель шукає структуру сама. Протилежність — навчання з учителем, де для кожного прикладу є правильна відповідь. *Приклад:* KMeans у модулі 04 не бачить, яка тема в кожного повідомлення.

Мовні моделі

- **Few-shot** (05) — Кілька готових прикладів прямо в запиті, щоб модель зрозуміла формат. Ваги не змінюються — модель «вчиться в контексті». Дешево і швидко, але приклади займають місце в запиті. *Приклад:* У запит додано 3 пари «текст → запитання».
- **GIFT** (05) — Текстовий формат запитань для імпорту в Moodle. Програма експортує згенерований і перевірений тест у GIFT і Word. У Moodle: «Банк запитань → Імпорт → GIFT». *Приклад:* ::Q1:: Що таке OSINT? { =Розвідка з відкритих джерел ~Тип мережі ~Канал зв'язку }
- **LoRA** (05) — Донавчання великої моделі через маленькі додаткові матриці; основні ваги заморожені. Навчається ≈ 1,1 млн ваг замість 600 млн (0,2 %). Адаптер — окремий маленький файл, який можна вмикати й вимикати. *Приклад:* Адаптер qwen3-lora навчився відповідати строго у форматі тесту.

- **Qwen3** (05) — Сімейство відкритих мовних моделей Alibaba (2025); у програмі — найменша, на 0,6 млрд ваг. Багатомовна, працює на звичайному ноутбуці (≈ 2,5 ГБ пам'яті). Має режим міркувань, який вимкнено для швидкості. *Приклад:* Qwen3-0.6B у модулі 05 працює в окремому процесі.
- **RAG** (05) — Генерація з пошуком: спершу знаходять найсхожіші приклади або документи, потім генерують відповідь. Пошук — за косинусною схожістю ембеддингів MiniLM. Модель отримує релевантний контекст і менше вигадує. *Приклад:* Для тексту про OSINT підбираються приклади саме про розвідку, а не про нейромережі.
- **Велика мовна модель** (05) — Модель, що генерує текст, передбачаючи наступне слово (токен) за попередніми. Навчена на величезних обсягах текстів. Може відповідати, перекладати, складати тести. Qwen3-0.6B — мала модель такого типу. *Приклад:* За текстом про маскування модель складає запитання з трьома варіантами.
- **Галюцинація** (05) — Впевнена, але хибна відповідь мовної моделі. Модель генерує правдоподібний текст, а не перевіряє факти. Тому згенеровані тести обов'язково переглядає викладач. *Приклад:* Модель вигадала «правильну» відповідь, якої немає в тексті.
- **Промпт** (05) — Інструкція для мовної моделі: що зробити і в якому форматі відповісти. Чітка інструкція з форматом (JSON) і прикладами помітно підвищує якість відповіді. *Приклад:* «Склади одне запитання з трьома варіантами. Відповідай тільки JSON».
- **Токен** (05) — Шматок тексту (слово або частина слова), з якими працює мовна модель. Українське слово часто ділиться на 2–4 токени. Довжину запиту й відповіді рахують у токенах. *Приклад:* «маскування» → «маск» + «ування».

Навчання з підкріпленням

- **PPO** (06) — Proximal Policy Optimization — популярний алгоритм RL, що змінює політику обережно, малими кроками. Обмеження (clip 0,2) не дає новій політиці сильно відрізнятися від старої за одне оновлення — навчання стабільне. *Приклад:* Готові агенти PPO для CartPole і Acrobot взято з rl-zoo (Hugging Face).
- **Агент** (06) — Той, хто приймає рішення в середовищі (програма-керувальник). Агент бачить стан, обирає дію, отримує винагороду і новий стан. Його «мозок» — нейронна мережа політики. *Приклад:* Агент PPO керує візком у CartPole.
- **Винагорода** (06) — Число, яким середовище оцінює дію агента; агент прагне максимізувати суму. У CartPole +1 за кожен крок, поки жердина стоїть (максимум 500). В Acrobot −1 за крок, поки не досягнуто мети (ближче до 0 — краще). *Приклад:* Середня винагорода 16 → 500 — агент знову втримує жердину весь епізод.
- **Епізод** (06) — Одна спроба агента від старту до завершення. Епізод CartPole завершується, коли жердина впала, візок виїхав за межі або минуло 500 кроків. *Приклад:* Оцінка «до» і «після» — 20 епізодів з однаковими seed.
- **Катастрофічне забування** (06) — Втрата старих навичок після донавчання на нових умовах. Перевіряється контрольною оцінкою в стандартному середовищі після донавчання. *Приклад:* Агент навчився довгій жердині, але почав падати з короткою.
- **Навчання з підкріпленням** (06) — Агент вчиться діяти методом спроб і помилок, отримуючи винагороду за добрі дії. Немає готових правильних відповідей — є лише оцінка результату. Агент сам знаходить стратегію, що максимізує винагороду. *Приклад:* Як дресирування: втримав жердину ще крок — +1 бал.
- **Політика** (06) — Правило агента: яку дію обрати в кожному стані. У PPO політика — нейронна мережа 4 → 64 → 64 → 2 дії. Донавчання змінює саме її. *Приклад:* Жердина

нахилилась вправо → штовхнути візок вправо.

- **Середовище** (06) — Симулятор, у якому діє агент: правила, фізика, винагороди. Зміна середовища (довша жердина, вітер, шум датчиків) імітує нові умови, до яких треба пристосуватися. *Приклад:* CartPole — візок із жердиною; Acrobot — дволанковий маятник.
- **Шум датчиків** (06) — Випадкова похибка вимірювань: агент бачить стан неточно. Імітує реальні сенсори. Агент, навчений на ідеальних даних, часто «ламається» — донавчання робить його стійкішим. *Приклад:* Кут жердини вимірюється з похибкою ±0,15.

15. Якщо щось пішло не так

Симптом	Що робити
INSTALL.bat зупинився з помилкою (обірвався Інтернет)	Запустіть INSTALL.bat ще раз: виконані кроки пропускаються, докачується лише відсутнє.
Помилка повторюється	Відкрийте `install.log` у папці TL-Lab. Повна переустановка бібліотек: `INSTALL.bat -Repair`.
«Шлях містить нелатинські символи»	Перенесіть папку в `C:\TL-Lab` або `D:\TL-Lab`.
Вікно не відкривається / «Сервер не відповів»	`TL-Lab-Stop.bat`, зачекайте 5 с, знову ярлик. Причина — в `app\logs\server.err.log`. Адреса вручну: http://127.0.0.1:8501
Після оновлення видно старий інтерфейс	`TL-Lab-Stop.bat`, потім ярлик; у браузері Ctrl+F5.
«Connection error» / сторінка не оновлюється	Сервер зупинився: запустіть ярлик знову і надішліть `app\logs\server.err.log` і `server.log`.
Модуль 05: «процес генерації аварійно завершився»	Програма працює далі. Закрийте зайві програми (Qwen потребує ≈ 2,5 ГБ ОЗП) і повторіть; деталі — `app\logs\qwen_worker.log`.
Модуль 03 довго перший раз	YAMNet (TensorFlow) стартує в окремому процесі до 20 с; далі працює швидко.
Модуль 06: «Observation spaces do not match»	Виправлено у версії 3.0 (агенти з Hugging Face підлаштовуються автоматично).
На вкладці «Адаптація» немає кнопки запуску	Виправлено у 3.1 (раніше її ховав режим «Доповідь»). У 3.0 — зніміть галочку «Режим «Доповідь»».
Модуль 03, експеримент: «розморожування не вдалося»	Версія YAMNet не дозволяє змінювати ваги. Звичайна адаптація збережена — результати коректні.
Фото чи звуки не завантажились	Програма працює й без них. Докачати: «Центр даних» → «Зібрати...» або повторний INSTALL.bat.
На картці модуля «немає моделі» чи «бракує бібліотек»	Повторно запустіть INSTALL.bat (потрібен Інтернет). Стан — «Налаштування → Система».
Клас «не навчається» / «мало даних»	Додайте фото (≥ 5, краще 20–30) чи записи (≥ 4, краще 10+) у «Центрі даних».
Програма гальмує на слабкому ПК	«Налаштування → Інтерфейс»: фон «Вимкнено», переходи вимкнути; одна задача одночасно.
Навчання «в черзі» і не починається	У режимі «1 задача» друга чекає на першу — це нормально.

16. Безпека даних

- У програмі використовуються **лише відкриті, синтетичні або знеособлені дані**: документи, повідомлення й тести вигадані («навчальна рота», «умовний противник»); фото — з Wikimedia Commons з вільними ліцензіями; звуки — ESC-50 і Wikimedia Commons. Джерела й ліцензії — у `app\data\SOURCES*.md`.
- Після автоматичного збору перегляньте фото й прослухайте записи; видаляйте знімки реальних позицій чи техніки ЗСУ, прив'язані до місцевості.
- Не завантажуйте в програму й у Colab службові документи чи матеріали реальної оперативної обстановки.
- Після встановлення програма працює **повністю офлайн** і нічого не надсилає в мережу.

17. Література

1. He K., Zhang X., Ren S., Sun J. Deep Residual Learning for Image Recognition. CVPR, 2016 (arXiv:1512.03385).
2. Selvaraju R. R. et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. ICCV, 2017.
3. Reimers N., Gurevych I. Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation. EMNLP, 2020.
4. Gemmeke J. F. et al. Audio Set: An ontology and human-labeled dataset for audio events. ICASSP, 2017.
5. Howard A. G. et al. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861, 2017.
6. Piczak K. J. ESC: Dataset for Environmental Sound Classification. ACM Multimedia, 2015.
7. Lloyd S. Least squares quantization in PCM. IEEE Trans. Information Theory, 1982 (KMeans).
8. Rousseeuw P. J. Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. J. Comput. Appl. Math., 1987.
9. van der Maaten L., Hinton G. Visualizing Data using t-SNE. JMLR, 2008.
10. Hu E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR, 2022 (arXiv:2106.09685).
11. Lewis P. et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS, 2020.
12. Qwen Team. Qwen3 Technical Report. arXiv:2505.09388, 2025.
13. Schulman J. et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
14. Barto A. G., Sutton R. S., Anderson C. W. Neuronlike adaptive elements that can solve difficult learning control problems. IEEE Trans. SMC, 1983.
15. Raffin A. et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. JMLR, 2021.

Навчальний макет. Не для бойового застосування.